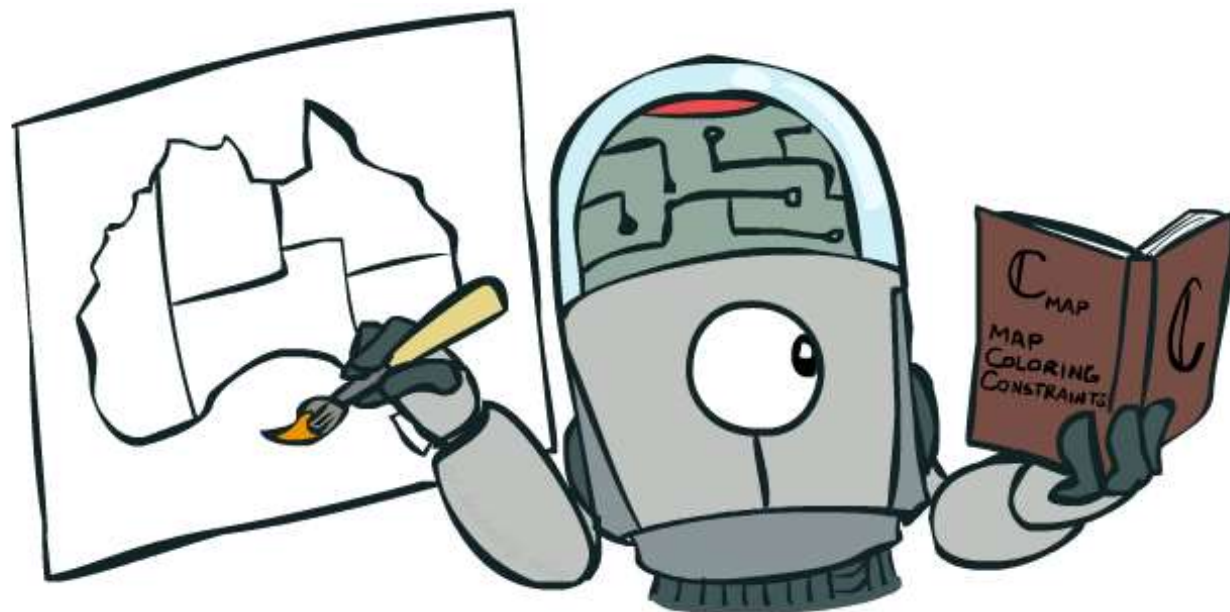


AIT2004: Cơ sở Trí tuệ nhân tạo

Bài toán thỏa mãn ràng buộc (CSP)



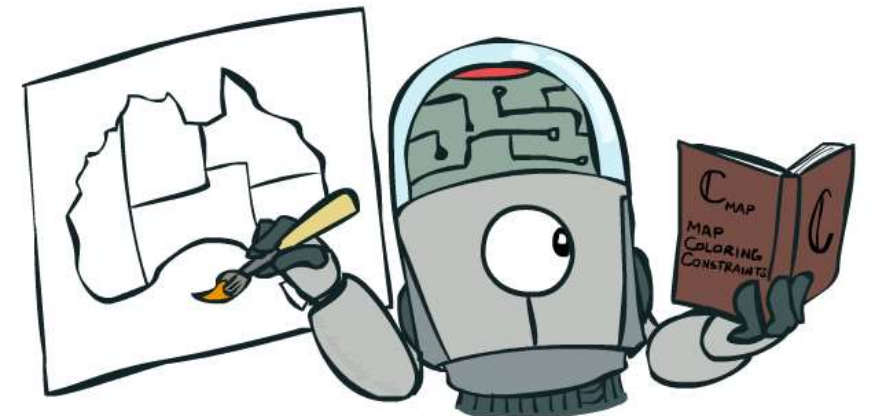
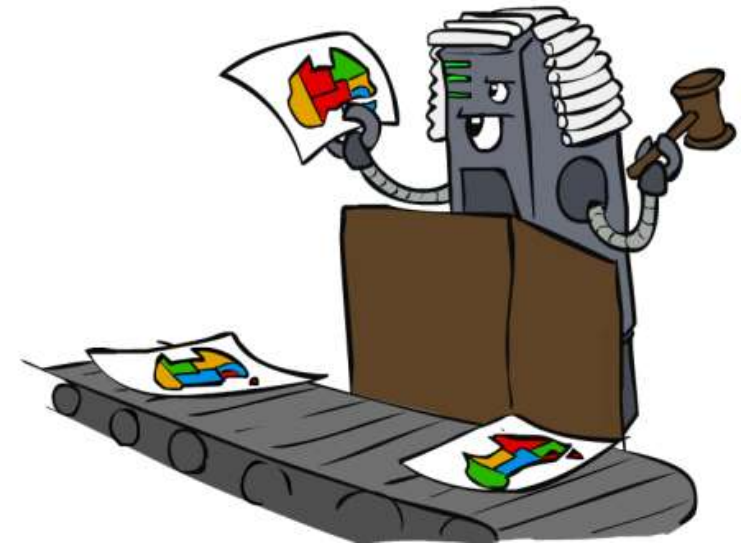
Sử dụng một số slide từ: CMU AI, <http://ai.berkeley.edu>

Học liệu

- Đinh Mạnh Tường, *Trí tuệ nhân tạo – Cách tiếp cận hiện đại*, NXB Khoa học kỹ thuật, 2024
 - Chương 6. Vấn đề thỏa mãn các ràng buộc
- Russell & Norvig, *AI: A Modern Approach*, 4th ed., Pearson 2020
 - Chapter 6. Constraint Satisfaction Problems

Bài toán thỏa mãn ràng buộc Constraint Satisfaction Problems

- CSP là một lớp các bài toán tìm kiếm đặc biệt
 - Hầu hết có tên riêng cho từng loại
 - Có thuật toán được chuyên biệt hóa
- Tìm kiếm tổng quát
 - Không có ràng buộc về trạng thái
 - Đích có thể là bất cứ trạng thái nào
 - Lời giải có thể là bất cứ cách nào đến đích
- Bài toán thỏa mãn ràng buộc (CSP)
 - Trạng thái được định nghĩa bằng tập biến X_i với giá trị thuộc miền D (D có thể phụ thuộc i)
 - Đích là tập giá trị của các biến thỏa mãn tập các ràng buộc



Ví dụ: tô màu bản đồ

- Làm thế nào để tô màu bản đồ bằng 3 màu mà các vùng cạnh nhau có màu khác nhau



Ví dụ: Tô màu bản đồ

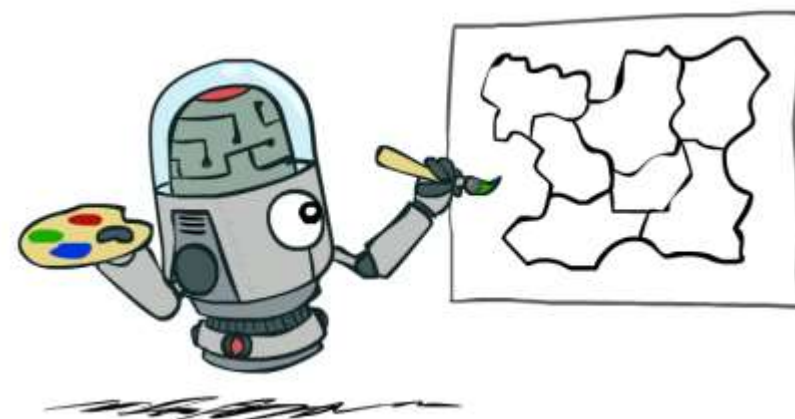
- Biến: WA, NT, Q, NSW, V, SA, T
- Miền giá trị: $D = \{\text{red, green, blue}\}$
- Ràng buộc: vùng cạnh nhau có màu khác nhau

Implicit: $WA \neq NT$

Explicit: $(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), \dots\}$

- Lời giải là tập giá trị thỏa mãn tất cả ràng buộc, ví dụ:

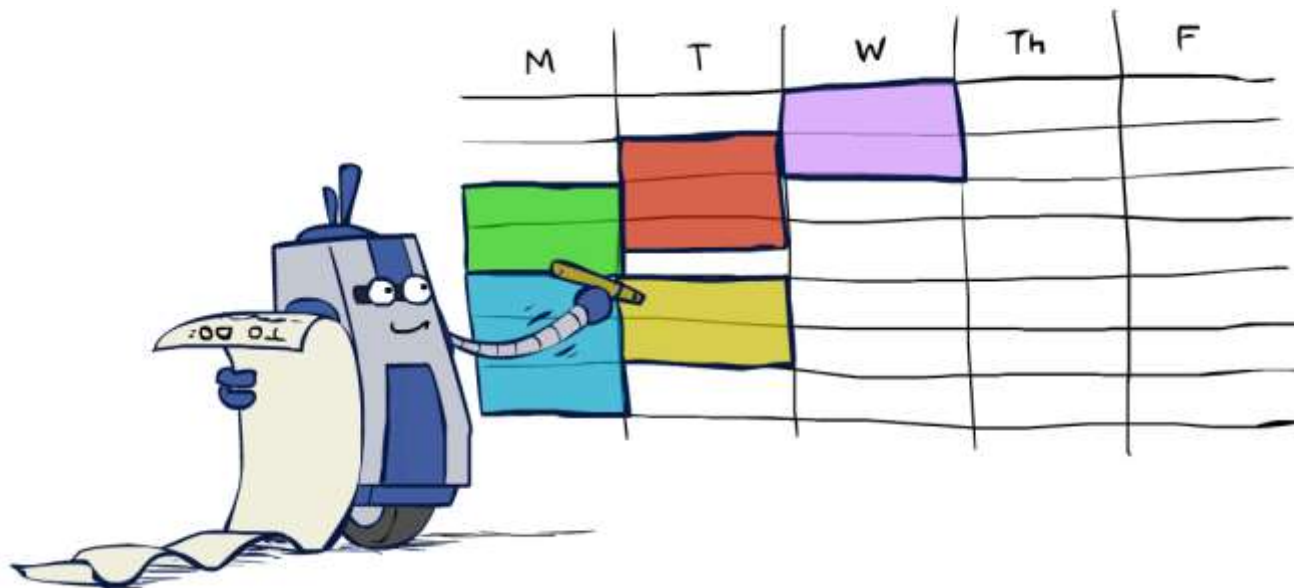
$\{WA=\text{red}, NT=\text{green}, Q=\text{red}, NSW=\text{green}, V=\text{red}, SA=\text{blue}, T=\text{green}\}$



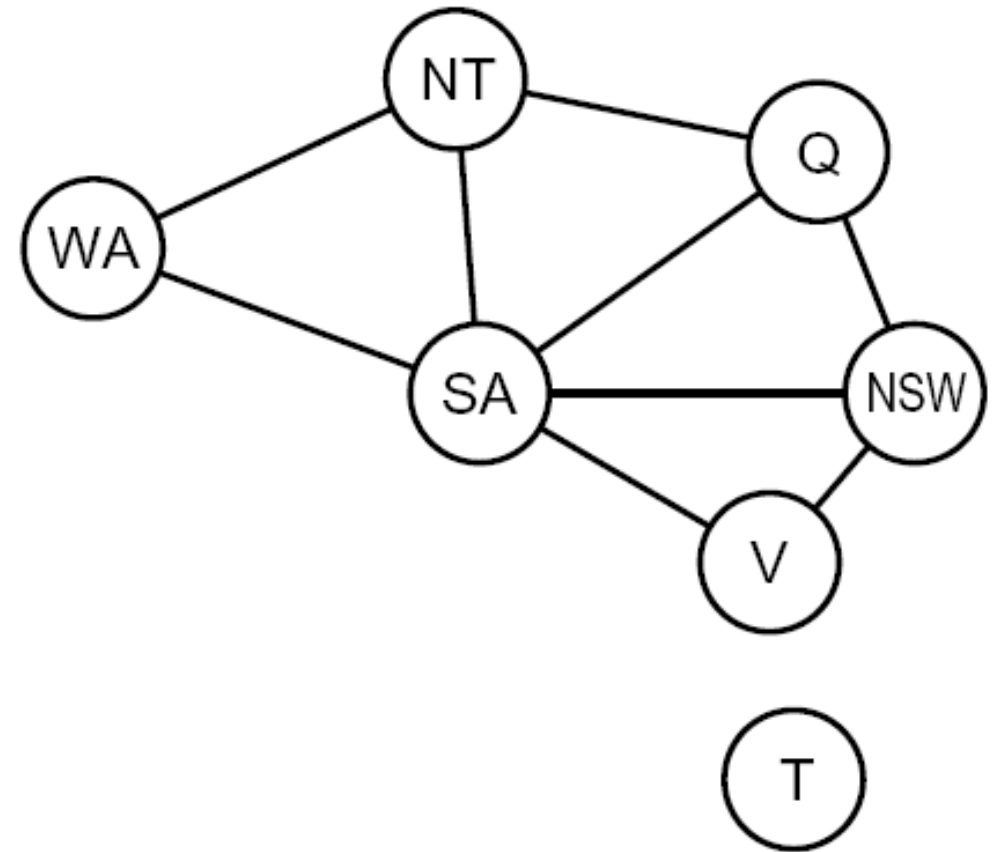
Tại sao cần tìm hiểu về CSP?

Nhiều bài toán thực tế có thể mô tả thành CSP

- Bài toán phân công: ví dụ, ai dạy lớp nào
- Lập thời gian biểu: ví dụ, lớp nào được dạy ở đâu, bao giờ?
- Cấu hình phần cứng
- Lập lịch vận chuyển, lịch sản xuất
- Thiết kế mạch điện tử
- Chẩn đoán lỗi
- ...

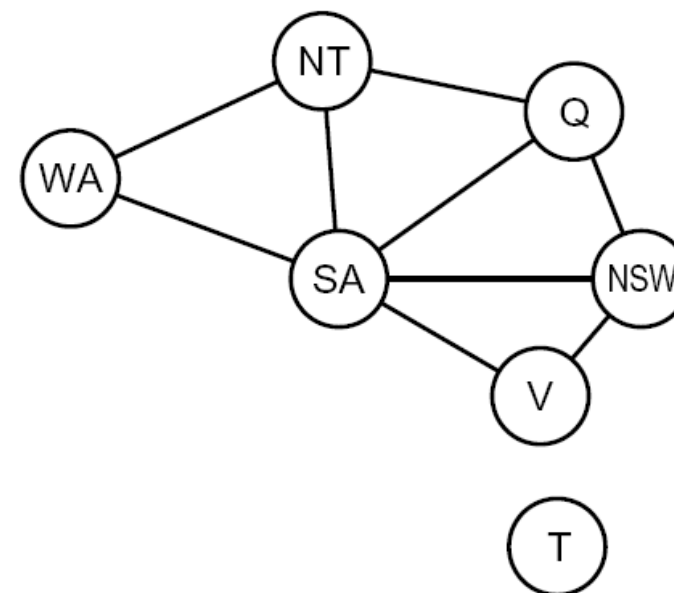


Biểu diễn bằng Đồ thị ràng buộc

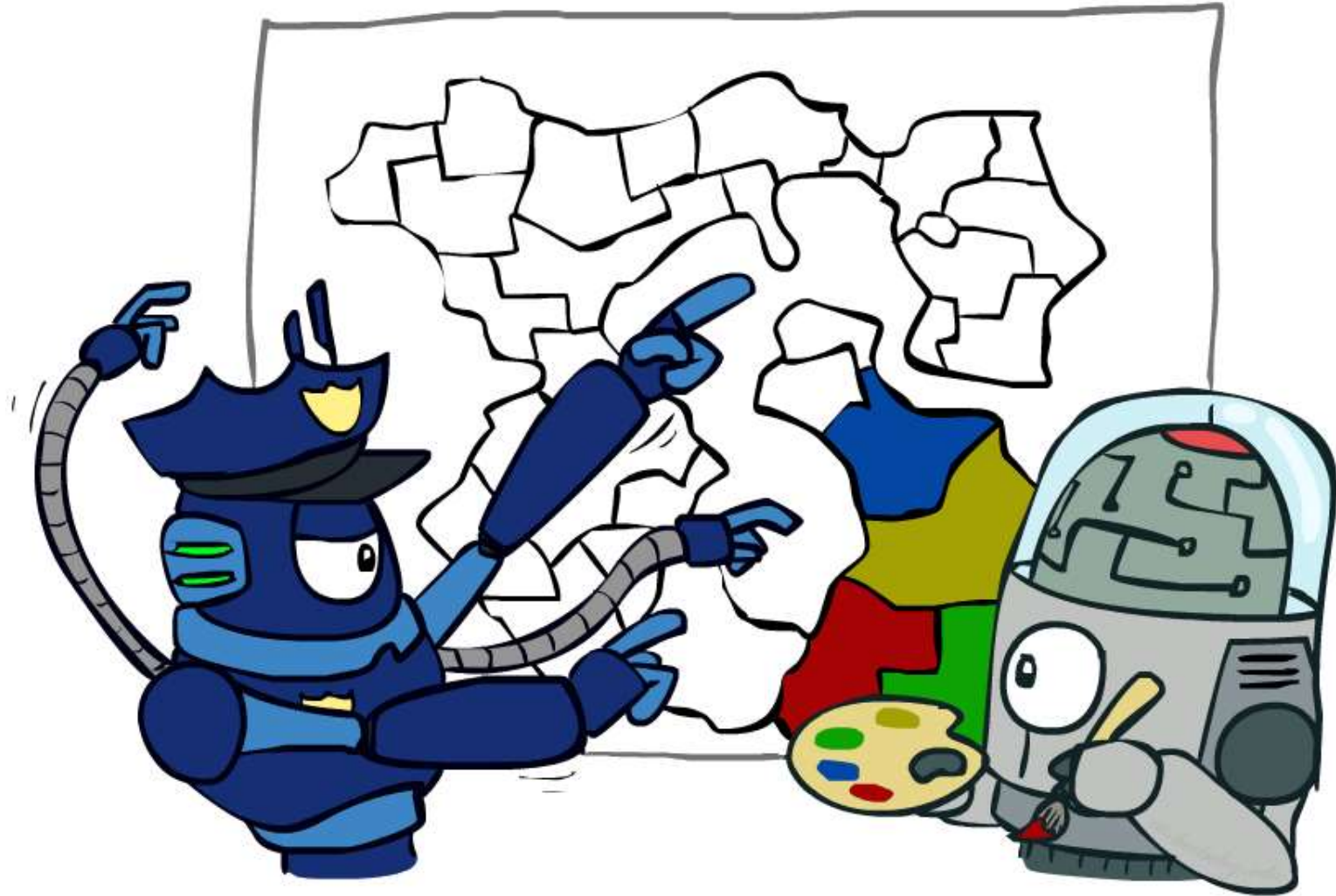


Đồ thị ràng buộc

- Ràng buộc nhị phân: mỗi ràng buộc liên quan nhiều nhất 2 biến
- Đồ thị ràng buộc nhị phân: nút là biến, cung thể hiện quan hệ ràng buộc
- Có thể khai thác cấu trúc đồ thị (đặc trưng của bài toán) để tăng tốc tìm kiếm
 - Ví dụ: giải quyết từng miền liên thông; hay cấu trúc cây (không có chu trình)



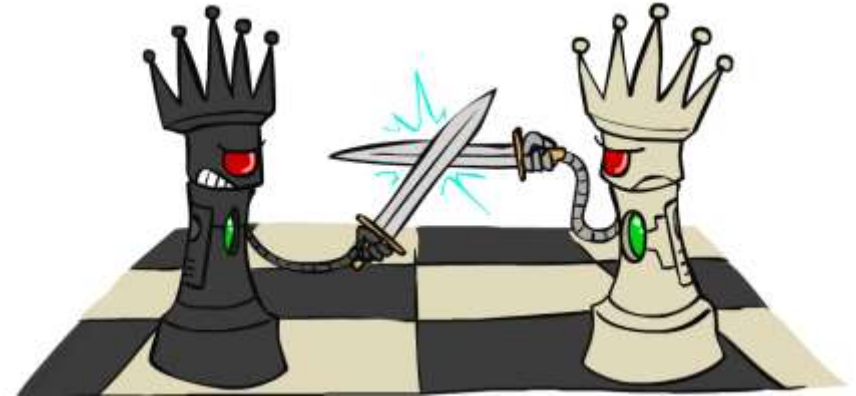
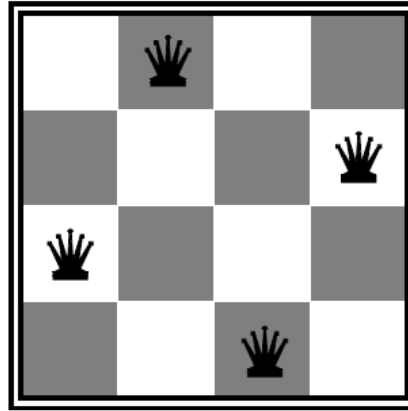
Một số dạng bài toán CSP



Ví dụ: N-Queens

- Mô hình hóa ô bàn cờ:

- Biến: X_{ij}
- Miền giá trị: $\{0, 1\}$
- Ràng buộc:



$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\sum_{i,j} X_{ij} = N$$

Ví dụ: N-Queens

- Mô hình hóa quân hậu:

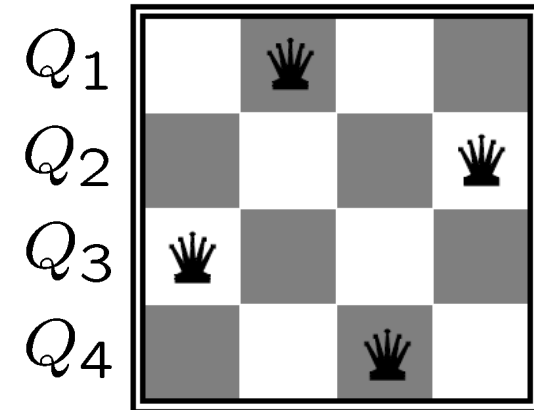
- Biến: Q_k
- Miền giá trị: $\{1, 2, 3, \dots, N\}$

- Ràng buộc:

Implicit: $\forall i, j \text{ non-threatening}(Q_i, Q_j)$

Explicit: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$

...



Ví dụ: Mật mã số học (Cryptarithmetic)

- Biến:

$F \ T \ U \ W \ R \ O \ X_1 \ X_2 \ X_3$

- Miền giá trị:

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- Ràng buộc:

$\text{alldiff}(F, T, U, W, R, O)$

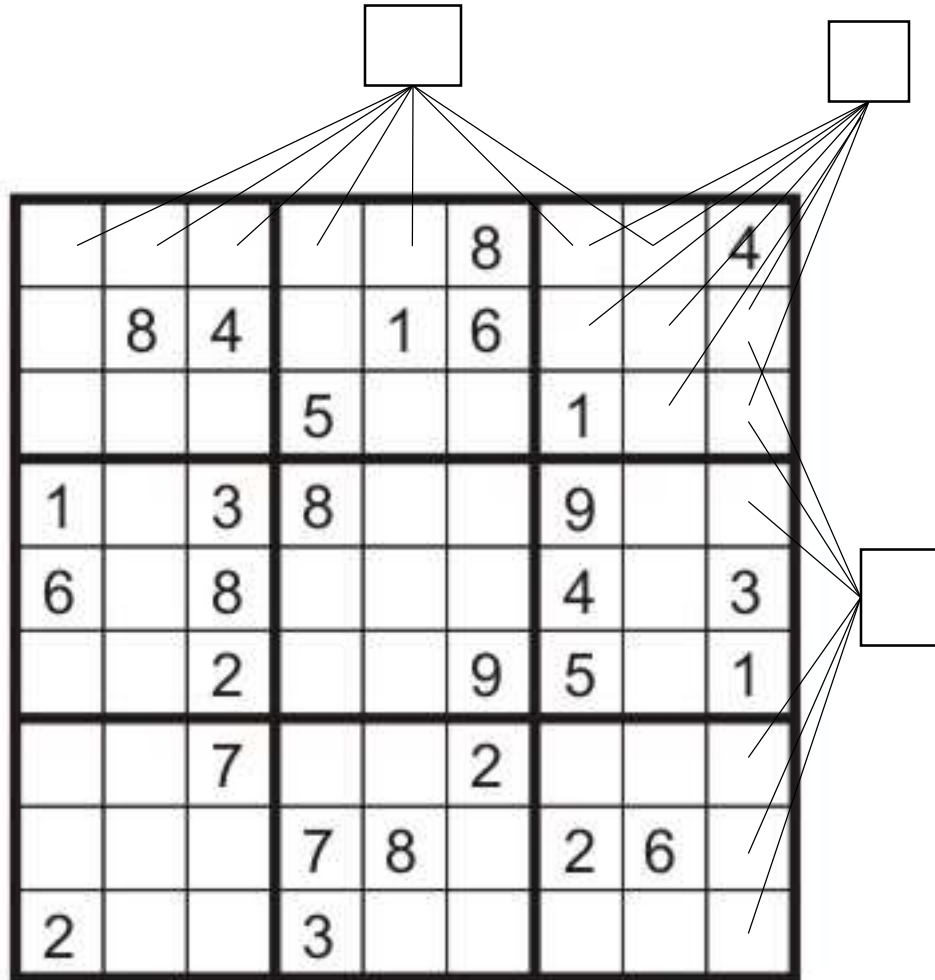
$O + O = R + 10 \cdot X_1$

$\dots$

	T	W	O
+	T	W	O
F	O	U	R



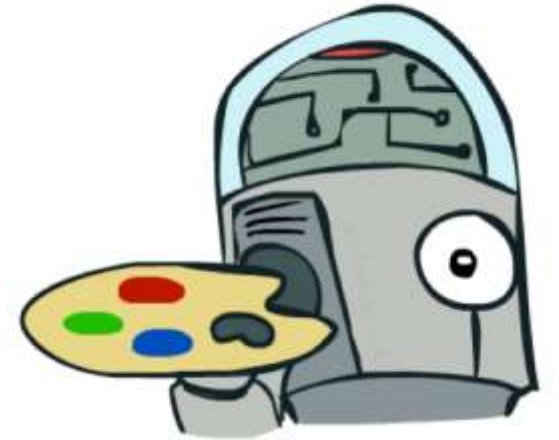
Ví dụ: Sudoku



- Biến: các ô trống
- Miền giá trị: $\{1,2,\dots,9\}$
- Ràng buộc:
 - 9-way alldiff for each column
 - 9-way alldiff for each row
 - 9-way alldiff for each region
 - (or can have a bunch of pairwise inequality constraints)

Các dạng CSPs

- Biến rời rạc
 - Miền hữu hạn
 - Tập giá trị d hữu hạn
 - Ví dụ, ràng buộc logic (Boolean CSPs)
 - Miền vô hạn (số tự nhiên,...)
 - Ví dụ, lập lịch với biến thời gian
 - Có thể giải được với ràng buộc tuyến tính
- Biến liên tục
 - Ví dụ giá trị vật lý
 - Có thể giải được ràng buộc tuyến tính với độ phức tạp đa thức



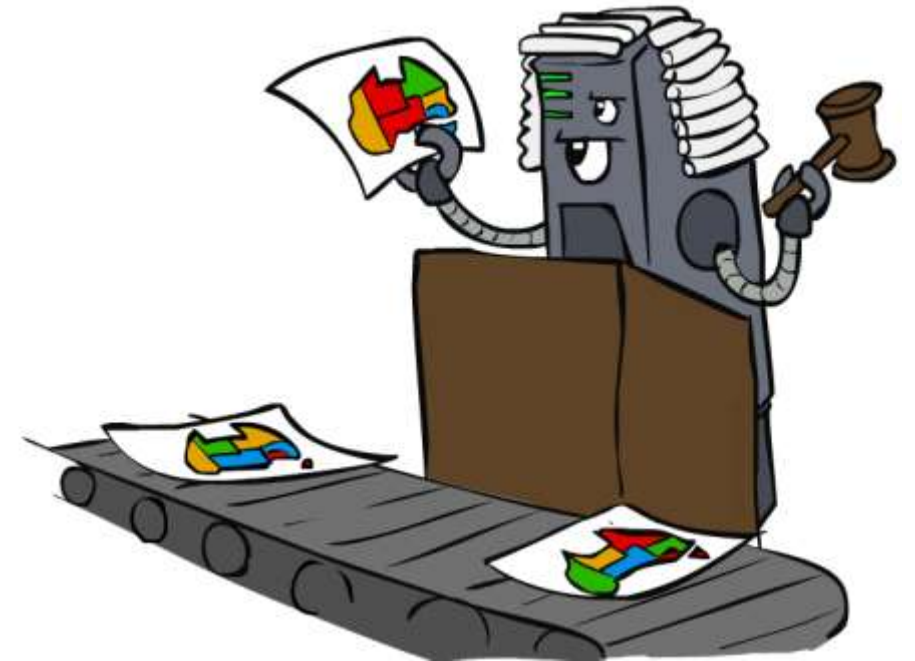
Các dạng ràng buộc

- Các dạng ràng buộc tường minh
 - Ràng buộc đơn (= giới hạn miền giá trị), ví dụ:
 $SA \neq \text{green}$
 - Ràng buộc nhị phân (2 biến), ví dụ:
 $SA \neq WA$
 - Ràng buộc bậc cao (nhiều hơn 2 biến), ví dụ
 $O + O = R + 10 \cdot X_1$
- Ràng buộc mềm (sở thích):
 - Ví dụ, màu đỏ thì tốt hơn màu xanh
 - Thường được biểu diễn bằng hàm đánh giá
 - Trở thành bài toán giải ràng buộc tối ưu



Giải bài toán CSPs

- Mô hình hóa thành bài toán tìm kiếm
- Trạng thái được định nghĩa bằng phép gán giá trị cho các biến
 - Trạng thái bắt đầu: tập gán rỗng $\{\}$
 - Chuyển trạng thái: gán giá trị cho một biến chưa được gán
 - Kiểm tra đích: phép gán đầy đủ và thỏa mãn mọi ràng buộc
- Áp dụng chiến lược tìm kiếm phù hợp
 - Vét cạn/quay lui
 - Tìm kiếm cục bộ (local search)
- Thu hẹp miền giá trị
 - Lan truyền ràng buộc



Giải CSP: Vét cạn/quay lui (backtracking search)

- Vét cạn/quay lui là thuật toán tìm kiếm cơ bản cho CSP, thực hiện tìm kiếm theo chiều sâu (DFS) với 2 cải tiến:

1) Chỉ gán giá trị cho một biến tại một thời điểm

- Thứ tự gán giá trị cho biến có thể hoán vị
 - [WA = red, sau đó NT = green] == [NT = green, sau đó WA = red]

2) Kiểm tra ràng buộc tại mỗi bước

- Chỉ sử dụng các giá trị không mâu thuẫn với các giá trị đã gán
 - Việc kiểm tra ràng buộc có thể cần “tính toán”
 - Là một hình thức tăng dần ràng buộc (incremental goal test)
- Có thể giải n-queens với $n \approx 25$

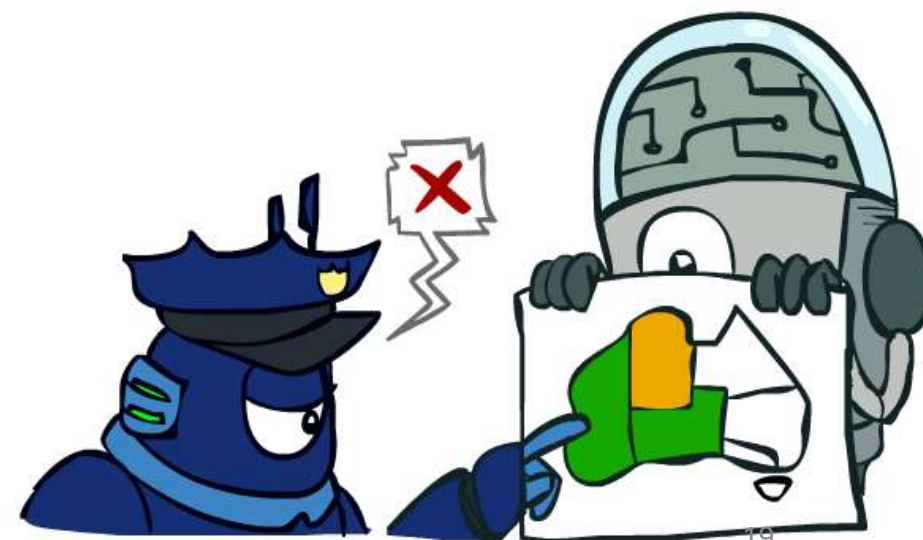
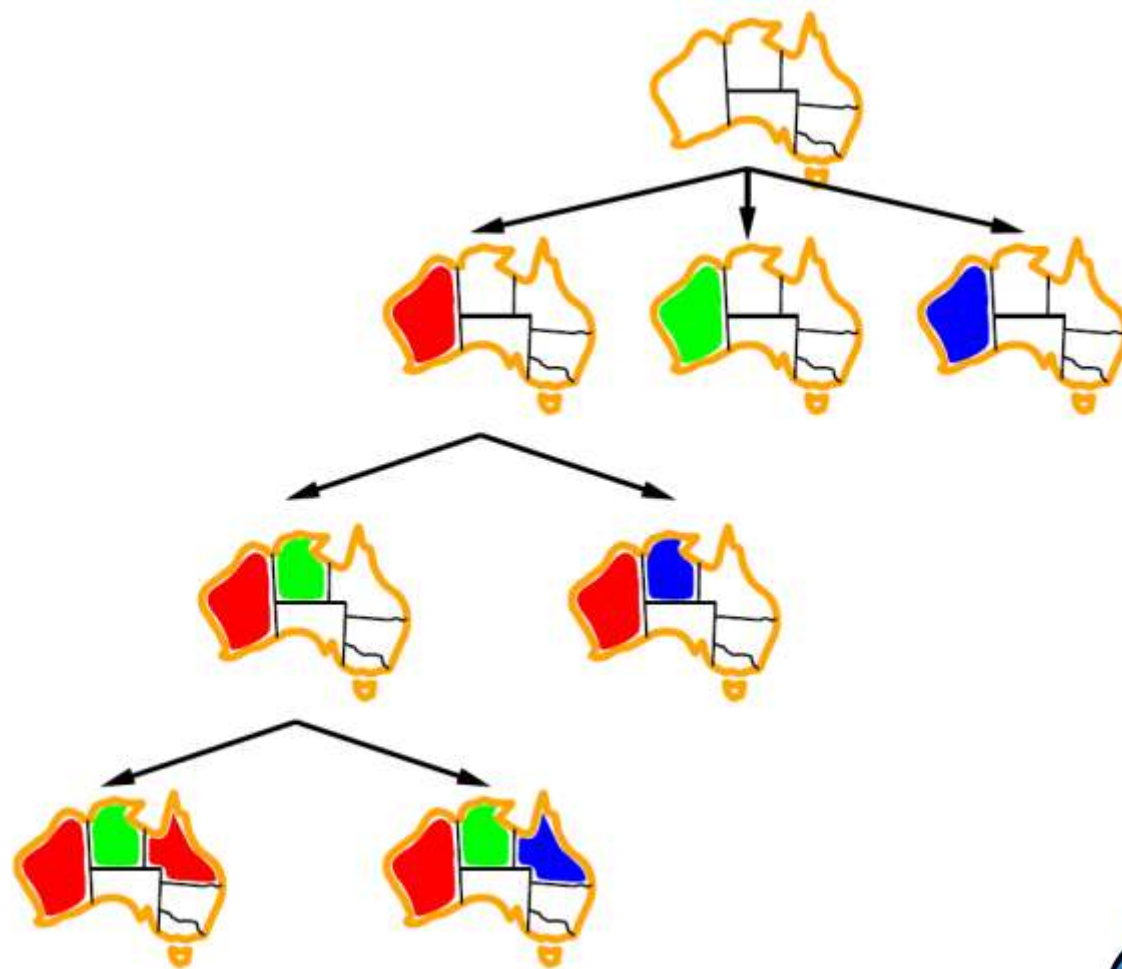


Thuật toán tìm kiếm Backtracking

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

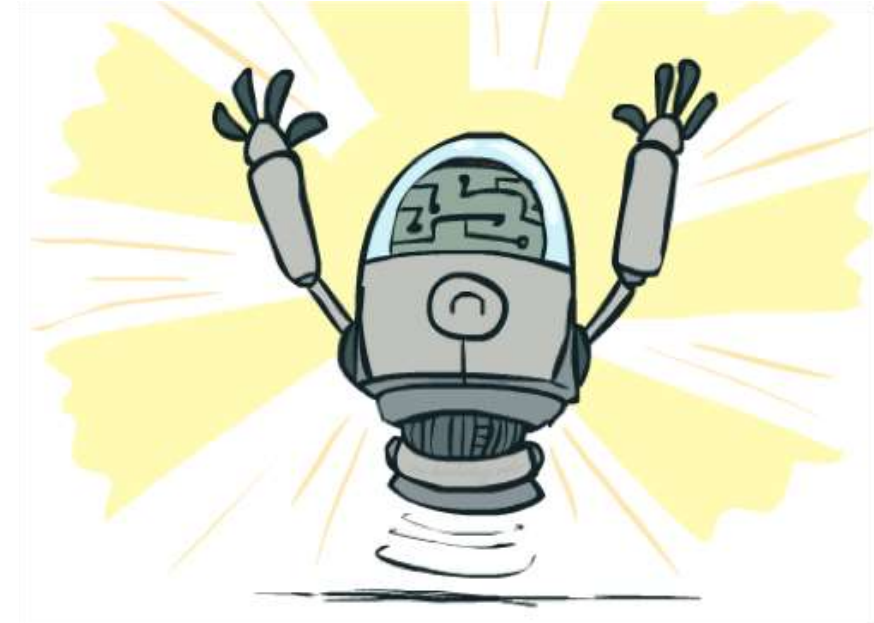
function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Ví dụ: Backtracking



Cải tiến cho CSP

- Vết cạn/quay lui tổng quát tốn thời gian
 - Có thể lặp đi lặp lại việc chọn các giá trị không phù hợp
- Lọc (Filtering): Có thể loại bỏ sớm các giá trị sẽ gây lỗi không?
- Điều chỉnh thứ tự:
 - Biến nào sẽ được gán tiếp theo?
 - Thử với thứ tự giá trị nào của biến?
- Tối ưu dựa trên cấu trúc đặc trưng bài toán



Lọc giá trị của biến

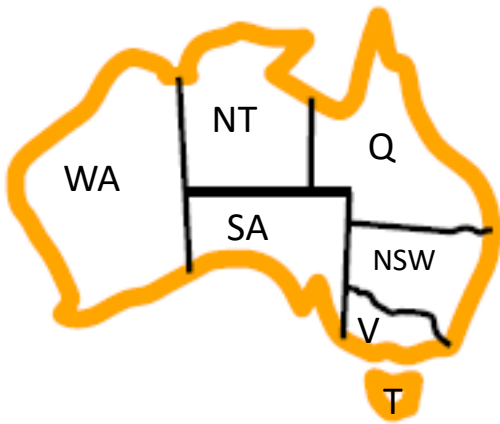


Lọc: Kiểm tra tiến (Forward Checking)

- Lọc: Kiểm tra miền giá trị của các biến chưa được gán và **loại bỏ** các giá trị không phù hợp.
- Kiểm tra tiến: Hình thức lọc đơn giản
 - Sau khi 1 biến được gán giá trị, kiểm tra các ràng buộc liên quan và loại bỏ các giá trị không phù hợp của các biến chưa được gán
 - Nếu một biến chưa gán không còn giá trị để gán thì **báo lỗi**

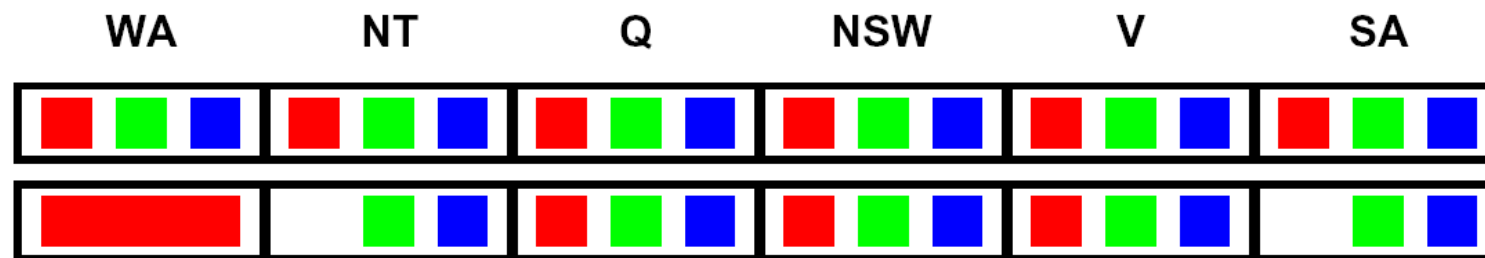
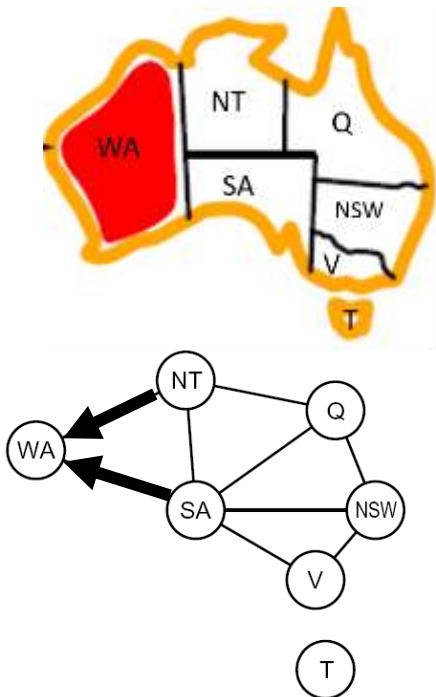
Filtering: Forward Checking

- Lọc: Kiểm tra miền giá trị của các biến chưa được gán và loại bỏ các giá trị không phù hợp.
- Kiểm tra tiến: Hình thức lọc đơn giản
 - Sau khi 1 biến được gán giá trị, kiểm tra các ràng buộc liên quan và loại bỏ các giá trị không phù hợp của các biến chưa được gán
 - Nếu một biến chưa gán không còn giá trị để gán thì **báo lỗi**



Filtering: Forward Checking

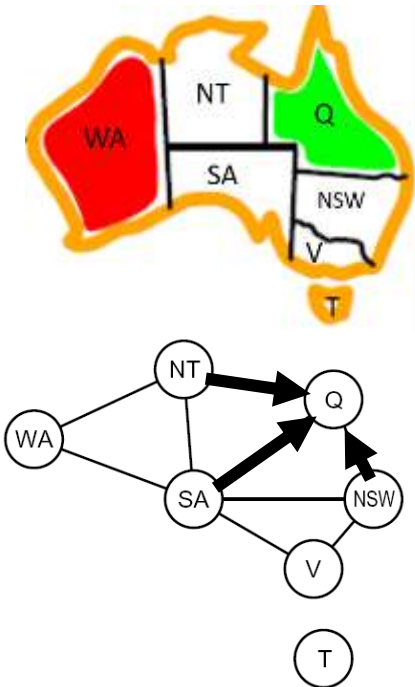
- Lọc: Kiểm tra miền giá trị của các biến chưa được gán và loại bỏ các giá trị không phù hợp.
- Kiểm tra tiến: Hình thức lọc đơn giản
 - Sau khi 1 biến được gán giá trị, kiểm tra các ràng buộc liên quan và loại bỏ các giá trị không phù hợp của các biến chưa được gán
 - Nếu một biến chưa gán không còn giá trị để gán thì **báo lỗi**



Ghi chú: Đồ thị nhị phân CSP đỉnh là biến, cạnh biểu diễn ràng buộc

Filtering: Forward Checking

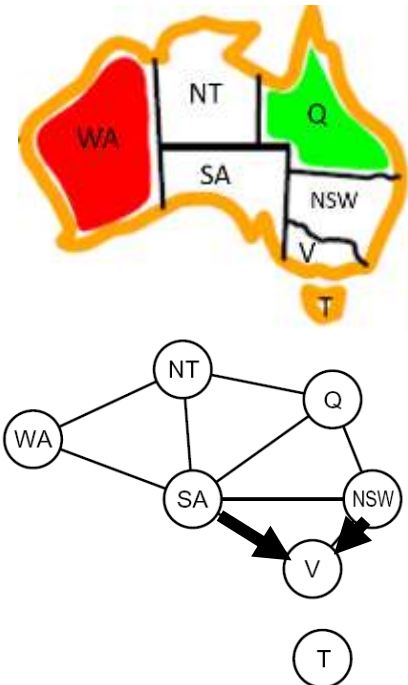
- Lọc: Kiểm tra miền giá trị của các biến chưa được gán và loại bỏ các giá trị không phù hợp.
- Kiểm tra tiến: Hình thức lọc đơn giản
 - Sau khi 1 biến được gán giá trị, kiểm tra các ràng buộc liên quan và loại bỏ các giá trị không phù hợp của các biến chưa được gán
 - Nếu một biến chưa gán không còn giá trị để gán thì **báo lỗi**



WA	NT	Q	NSW	V	SA

Filtering: Forward Checking

- Lọc: Kiểm tra miền giá trị của các biến chưa được gán và loại bỏ các giá trị không phù hợp.
- Kiểm tra tiến: Hình thức lọc đơn giản
 - Sau khi 1 biến được gán giá trị, kiểm tra các ràng buộc liên quan và loại bỏ các giá trị không phù hợp của các biến chưa được gán
 - Nếu một biến chưa gán không còn giá trị để gán thì **báo lỗi**

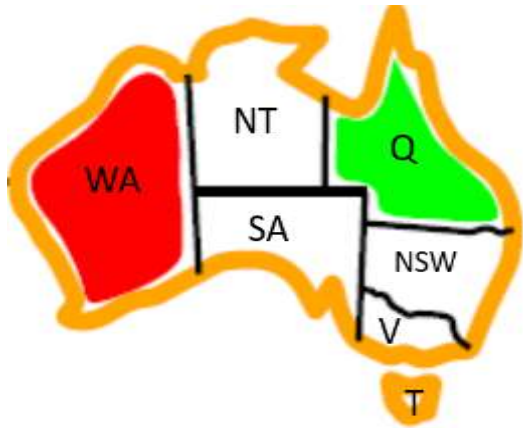


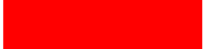
WA	NT	Q	NSW	V	SA

Lỗi – biến không còn giá trị để gán

Filtering: Constraint Propagation (lan truyền ràng buộc)

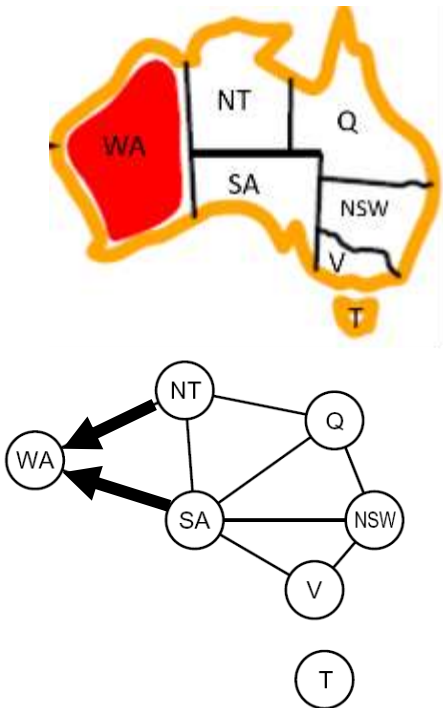
- Hạn chế của forward checking đơn giản: chuyển thông tin từ biến được gán tới biến chưa gán, nhưng chưa đủ “sâu” để phát hiện tất cả lỗi
 - Ví dụ, chưa phát hiện ra miền giá trị của NT và SA chỉ còn cùng màu xanh (blue)!
- *Constraint propagation*: từ ràng buộc này suy ra kết quả của ràng buộc khác



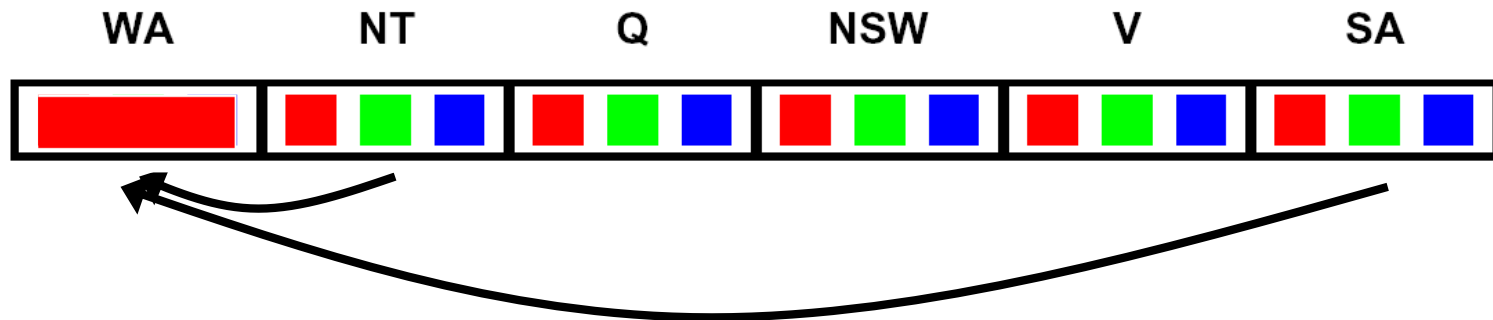
WA	NT	Q	NSW	V	SA
  	  	  	  	  	  
	 	  	  	  	 
			 	  	

Khái niệm Phù hợp cạnh (Arc Consistency)

- Cạnh $X \rightarrow Y$ là phù hợp nếu với mỗi x thuộc miền X tồn tại y thuộc miền Y thỏa mãn ràng buộc
- **Làm phù hợp cạnh:** Loại bỏ giá trị thuộc miền X mà không tồn tại giá trị phù hợp thuộc Y
- Forward checking: Chỉ làm phù hợp cạnh $X \rightarrow Y, \forall (X, Y) \in E$ với Y được gán mới

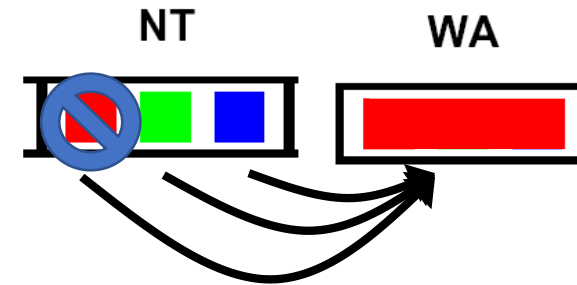
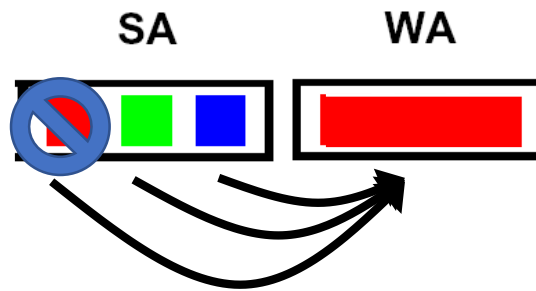
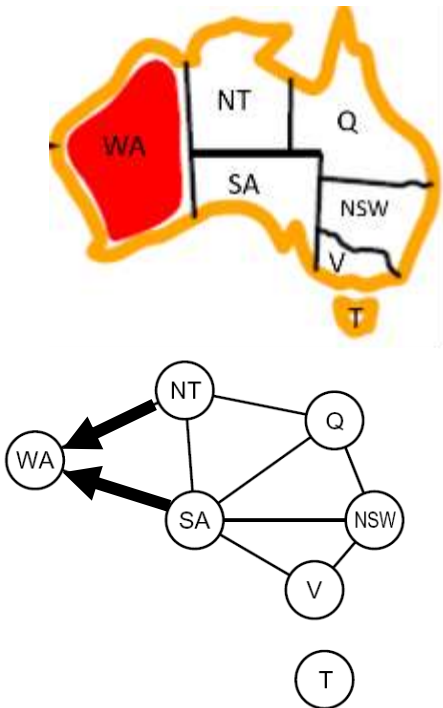


(Loại bỏ giá trị ở nút gốc!)



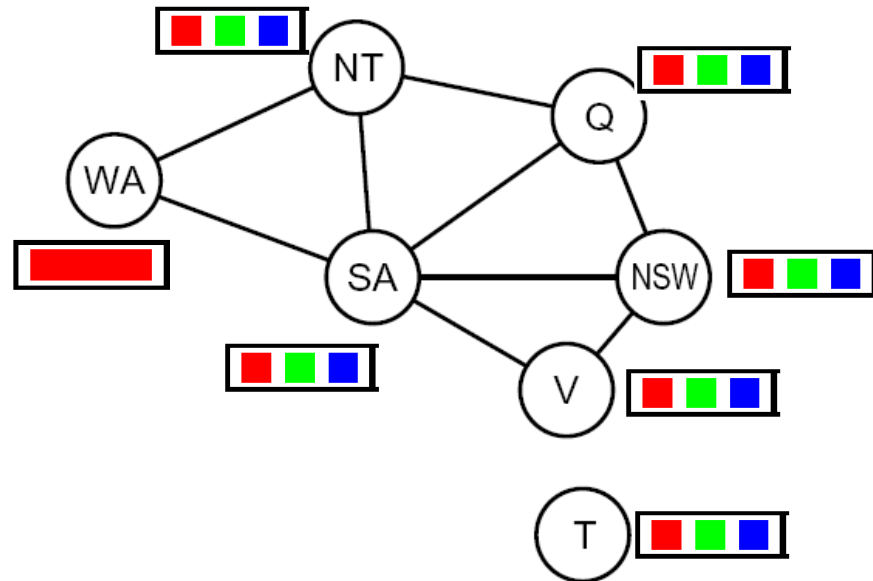
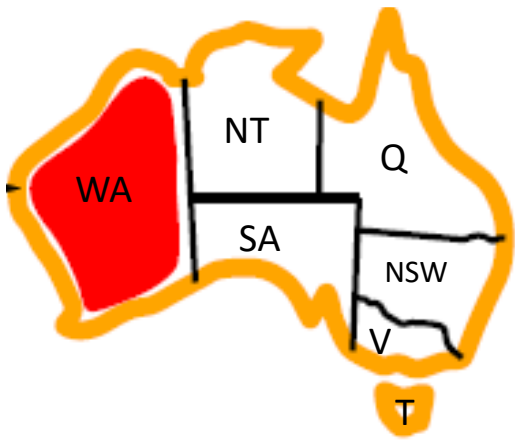
Phù hợp cạnh (Arc Consistency)

- Cạnh $X \rightarrow Y$ là phù hợp nếu với mỗi x thuộc miền X tồn tại y thuộc miền Y thỏa mãn ràng buộc
- **Làm phù hợp cạnh:** Loại bỏ giá trị thuộc miền X mà không tồn tại giá trị phù hợp thuộc Y
- Forward checking: Chỉ làm phù hợp cạnh $X \rightarrow Y, \forall (X, Y) \in E$ với Y được gán mới



Làm sao để làm phù hợp cạnh cho cả đồ thị CSP

- Ý tưởng: Duyệt mọi cạnh và làm phù hợp cạnh, lặp lại cho đến khi không có miền giá trị của đỉnh nào bị thay đổi trong cả một chu trình duyệt.
- AC-3 (Arc Consistency Algorithm #3): Thuật toán hiệu quả bỏ qua các ràng buộc mà miền giá trị không bị thay đổi từ lần duyệt trước.



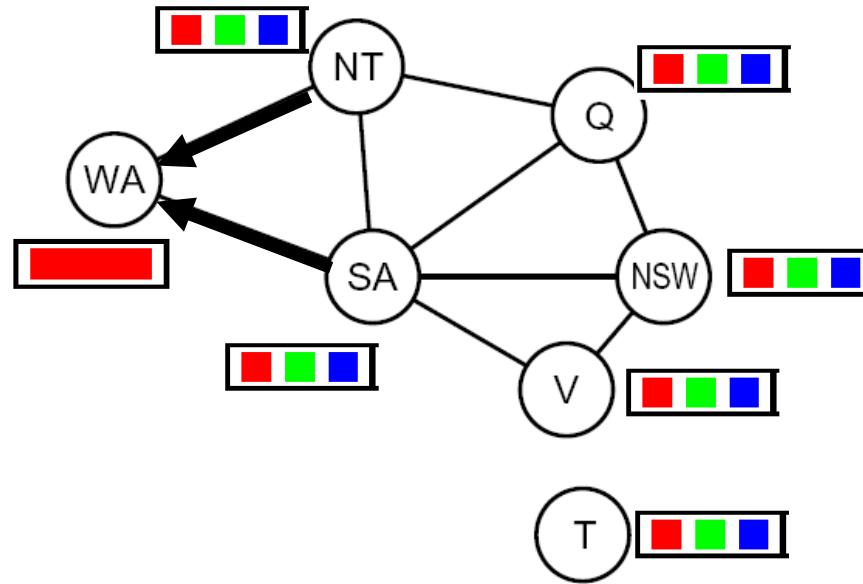
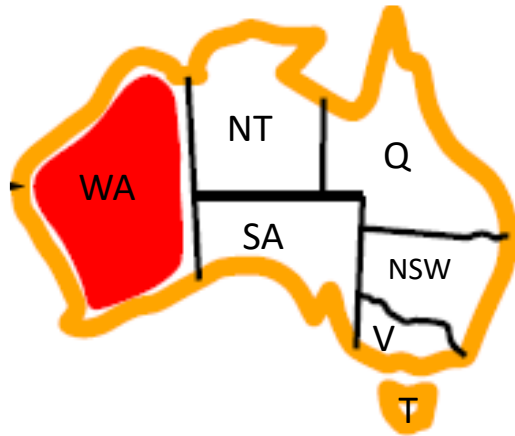
AC-3: Enforce Arc Consistency of Entire CSP

```
function AC-3(csp) returns the CSP, possibly with reduced domains
  inputs: csp, a binary CSP with variables  $\{X_1, X_2, \dots, X_n\}$ 
  local variables: queue, a queue of arcs, initially all the arcs in csp
  while queue is not empty do
     $(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$ 
    if REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) then
      for each  $X_k$  in NEIGHBORS[ $X_i$ ] do
        add  $(X_k, X_i)$  to queue

function REMOVE-INCONSISTENT-VALUES( $X_i, X_j$ ) returns true iff succeeds
  removed  $\leftarrow$  false
  for each  $x$  in DOMAIN[ $X_i$ ] do
    if no value  $y$  in DOMAIN[ $X_j$ ] allows  $(x, y)$  to satisfy the constraint  $X_i \leftrightarrow X_j$ 
      then delete  $x$  from DOMAIN[ $X_i$ ]; removed  $\leftarrow$  true
  return removed
```

Constraint Propagation!

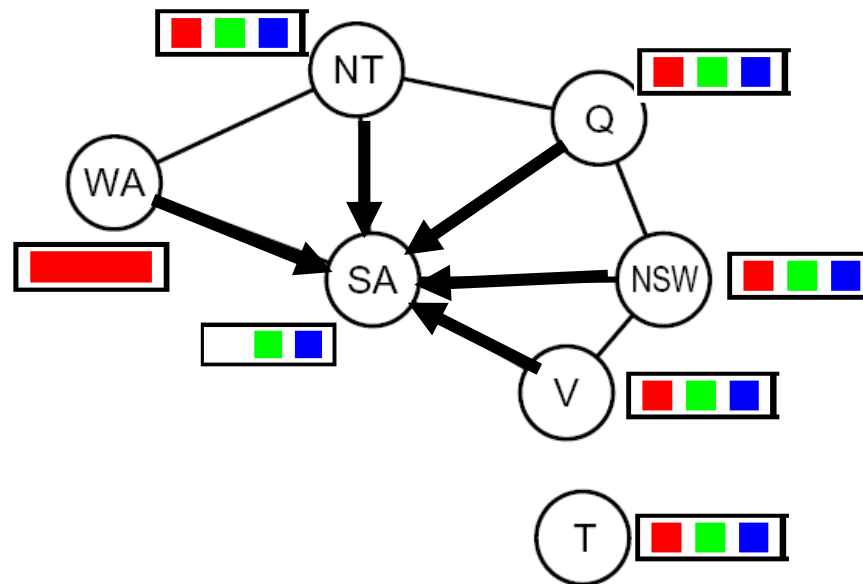
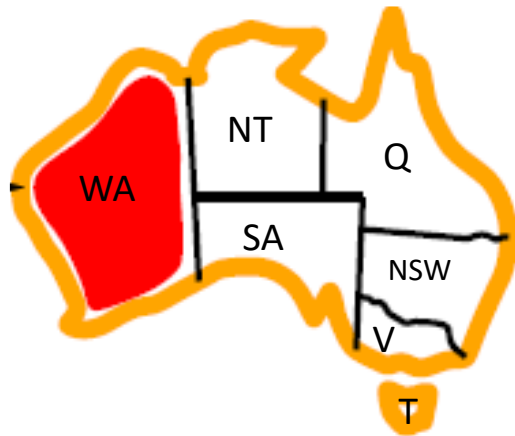
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
SA->WA
NT->WA

Remember: Delete from the tail!

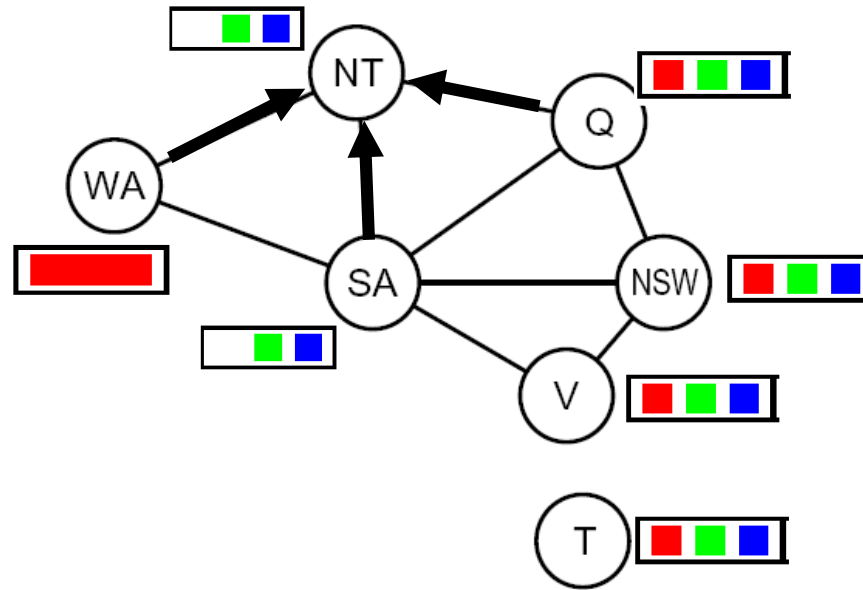
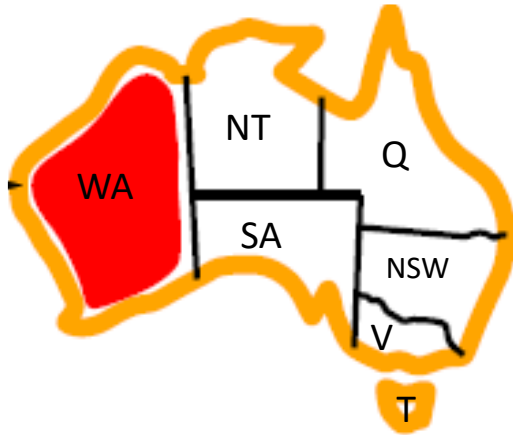
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
NT->WA
WA->SA
NT->SA
Q->SA
NSW->SA
V->SA

Remember: Delete from the tail!

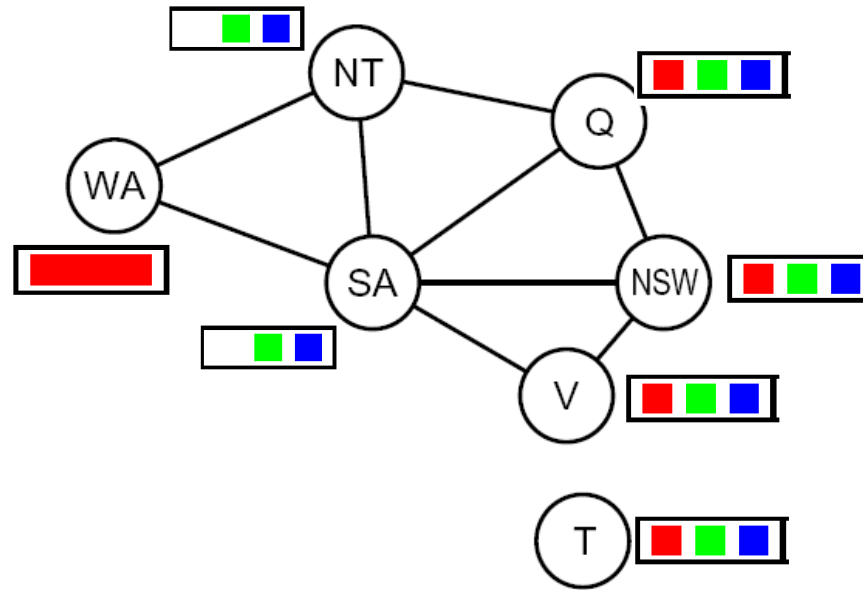
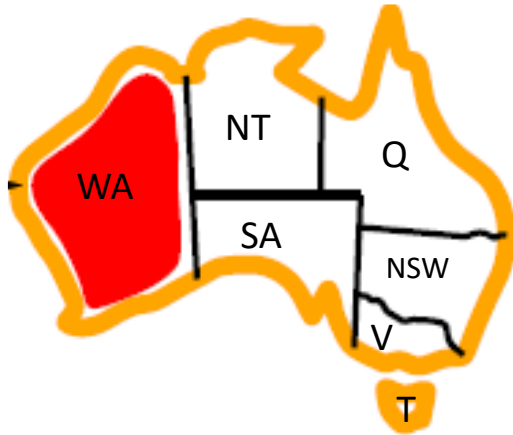
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
WA->SA
NT->SA
Q->SA
NSW->SA
V->SA
WA->NT
SA->NT
Q->NT

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

WA->SA

NT->SA

Q->SA

NSW->SA

V->SA

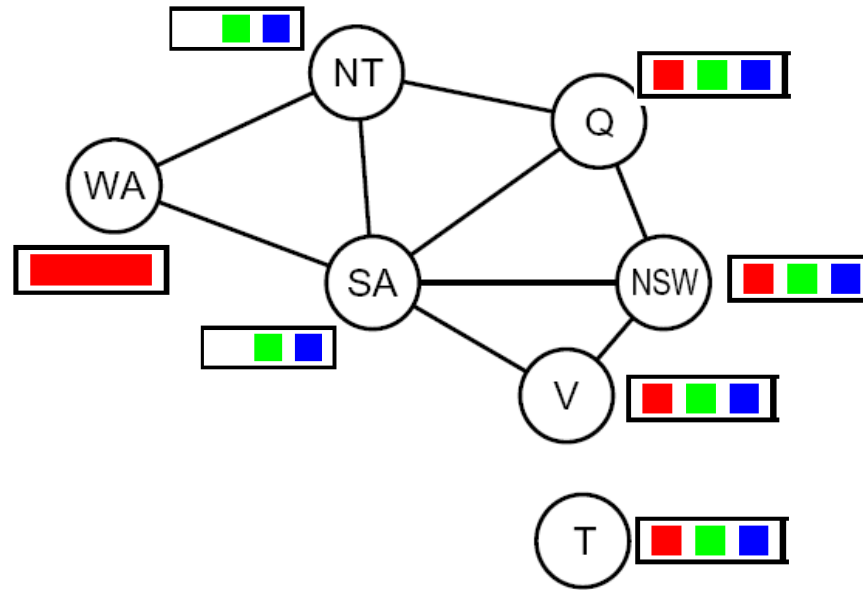
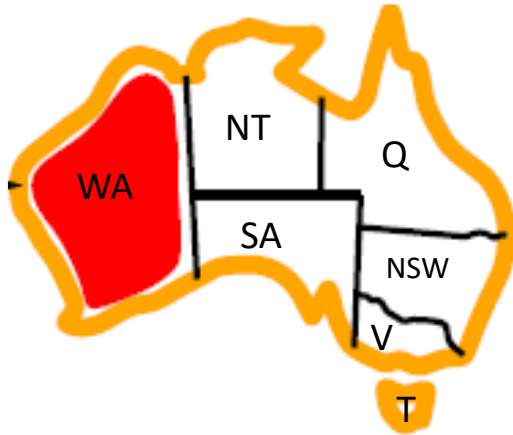
WA->NT

SA->NT

Q->NT

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

NT->SA

Q->SA

NSW->SA

V->SA

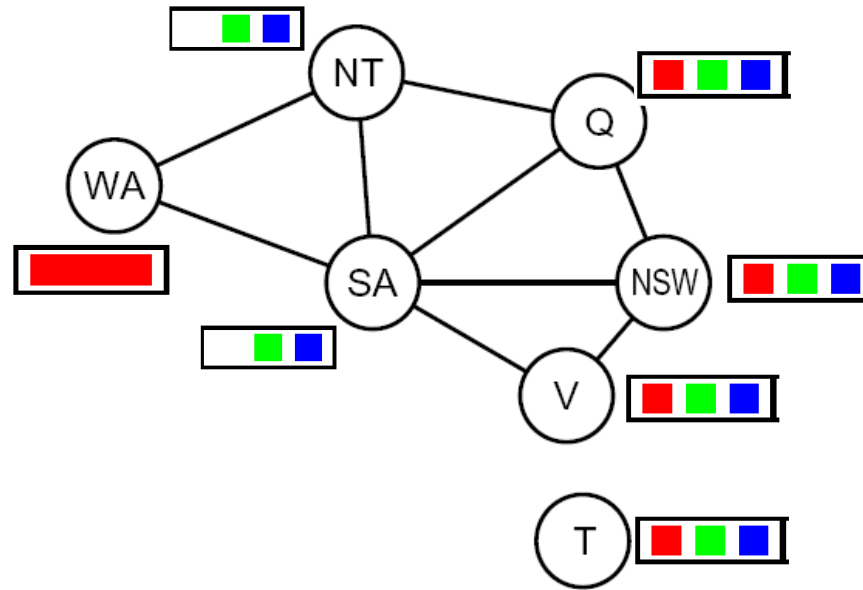
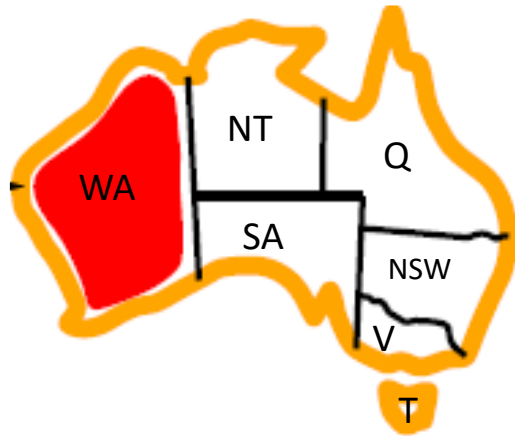
WA->NT

SA->NT

Q->NT

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

Q->SA

NSW->SA

V->SA

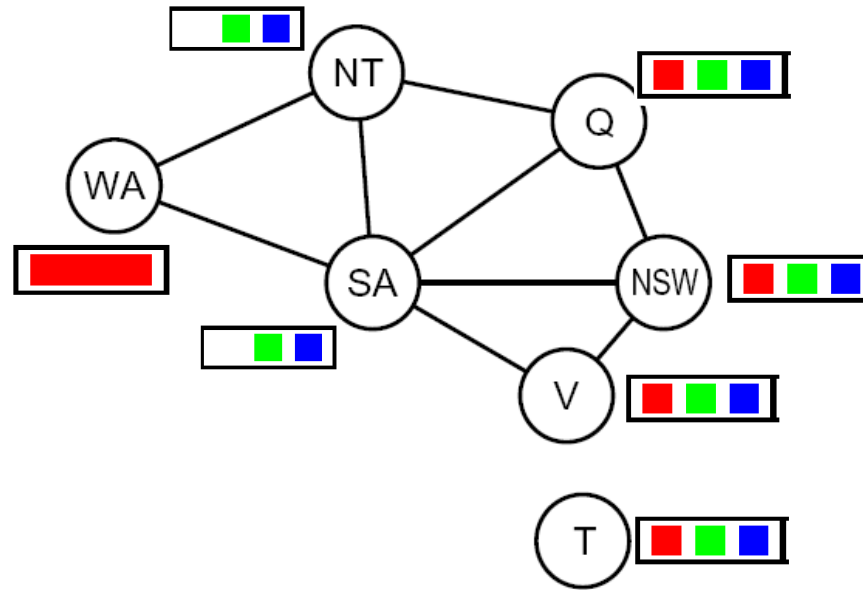
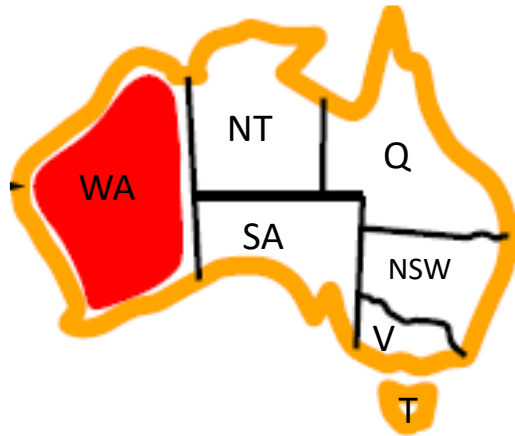
WA->NT

SA->NT

Q->NT

Remember: Delete from the tail!

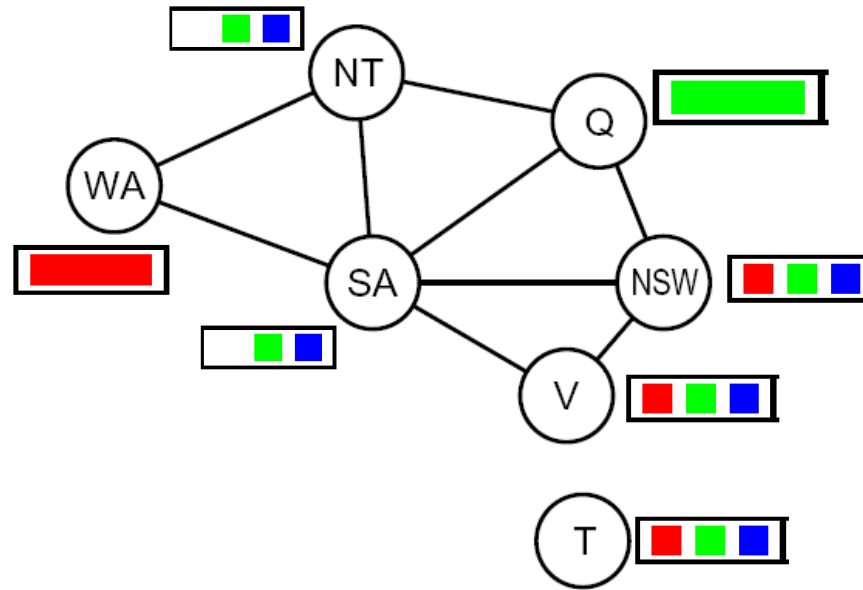
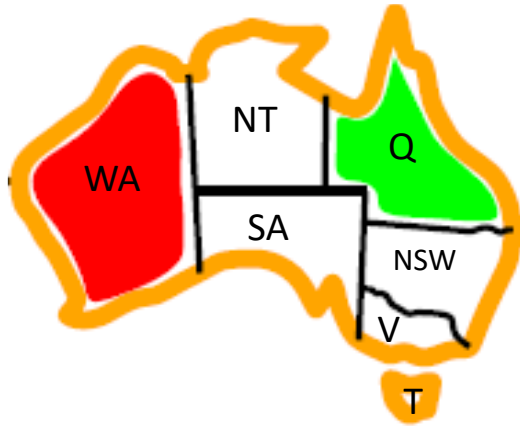
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
NSW->SA
V->SA
WA->NT
SA->NT
Q->NT

Remember: Delete from the tail!

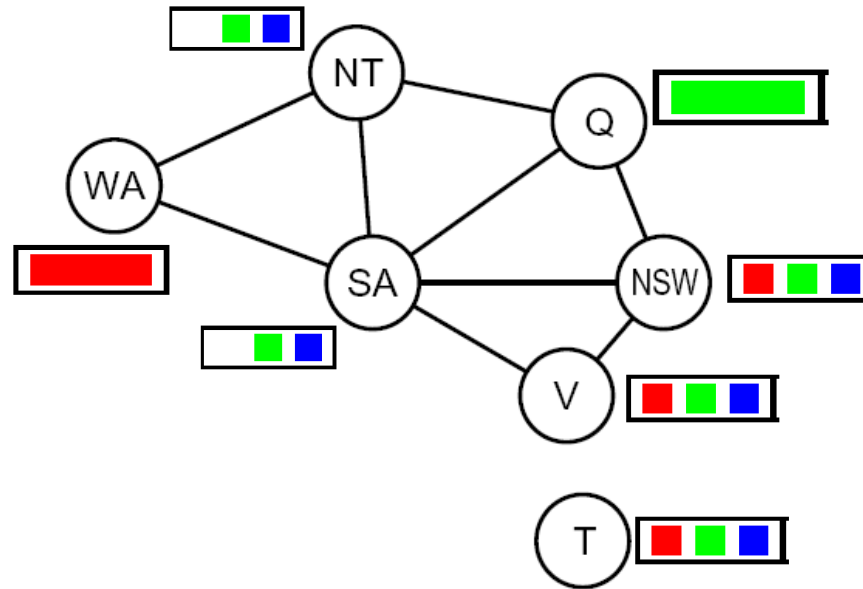
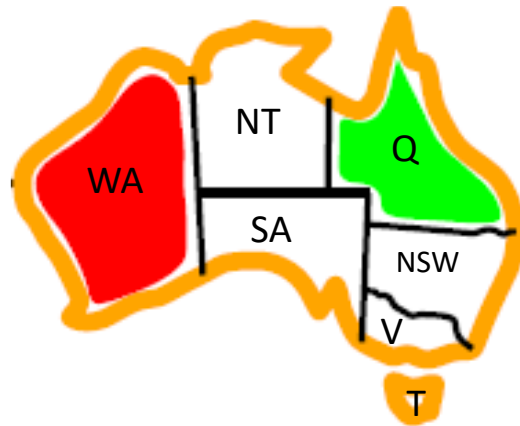
AC-3: Enforce Arc Consistency of Entire CSP



Queue:

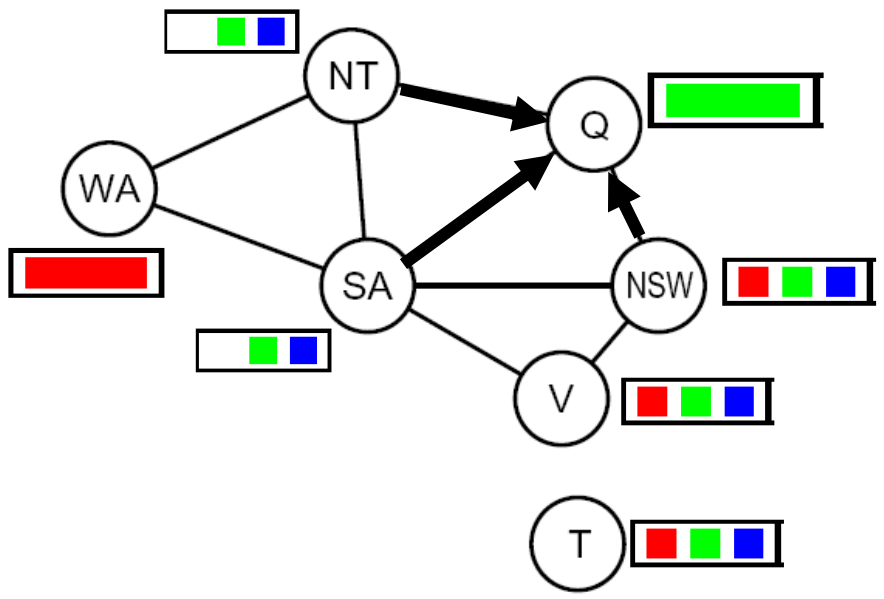
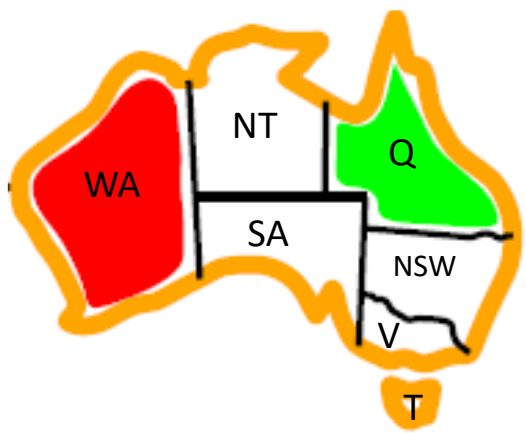
Remember: Delete from the tail!

After assigning Q to Green,
what gets added to the Queue?



Queue:

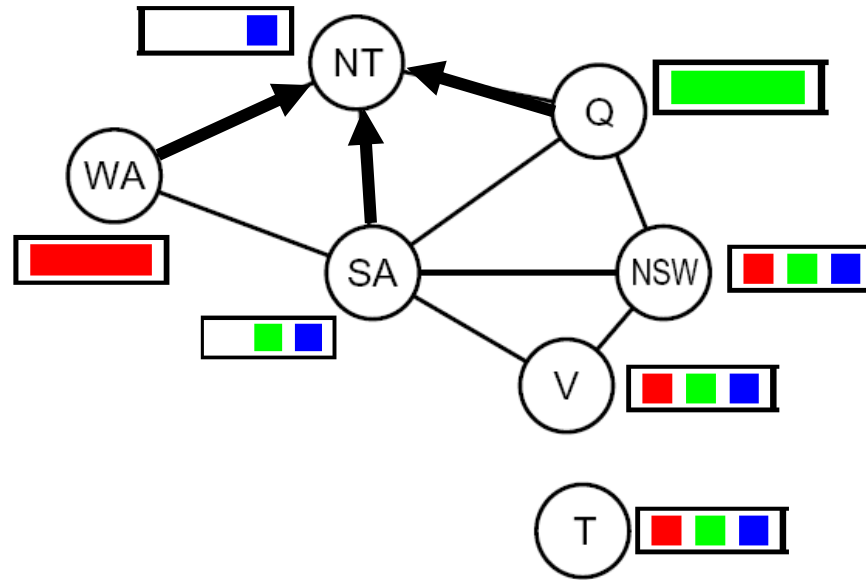
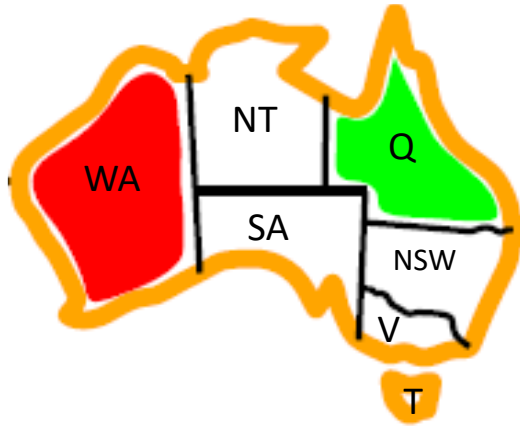
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
NT->Q
SA->Q
NSW->Q

Remember: Delete from the tail!

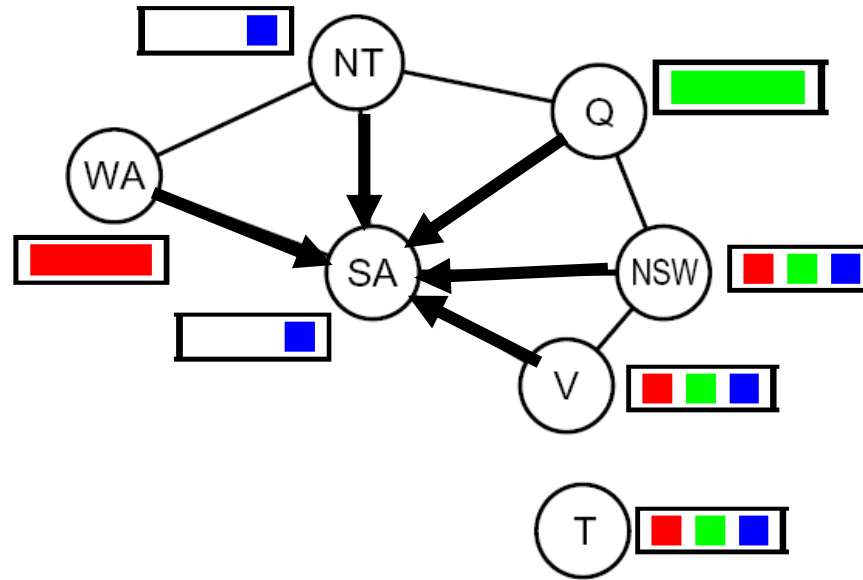
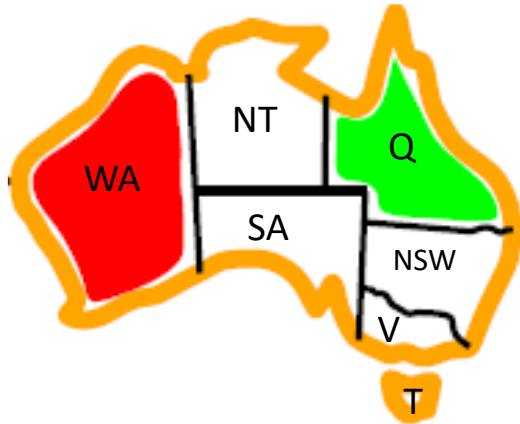
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
SA->Q
NSW->Q
WA->NT
SA->NT
Q->NT

Remember: Delete from the tail!

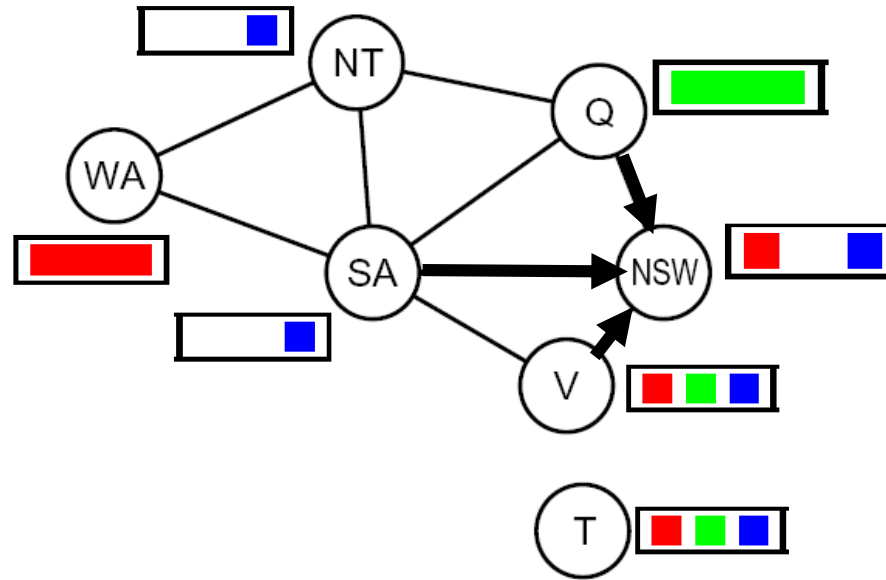
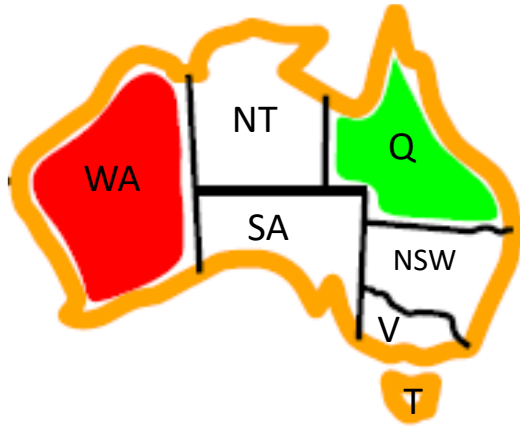
AC-3: Enforce Arc Consistency of Entire CSP



Queue:
NSW->Q
WA->NT
SA->NT
Q->NT
WA->SA
NT->SA
Q->SA
NSW->SA
V->SA

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

WA->NT

SA->NT

Q->NT

WA->SA

NT->SA

Q->SA

NSW->SA

V->SA

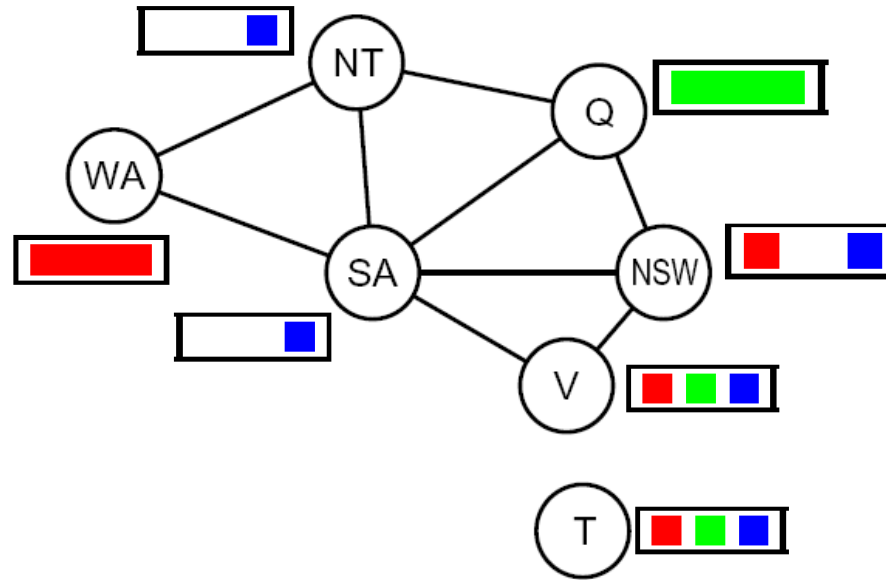
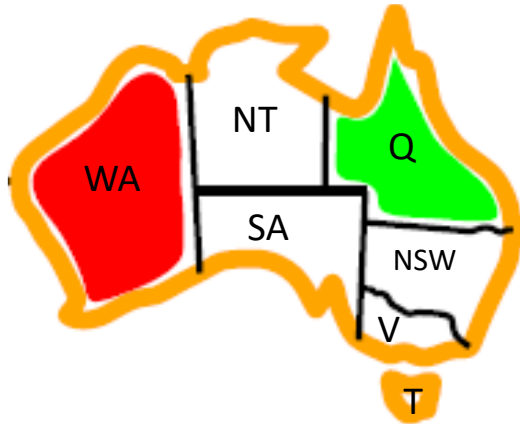
V->NSW

Q->NSW

SA->NSW

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

WA->NT

SA->NT

Q->NT

WA->SA

NT->SA

Q->SA

NSW->SA

V->SA

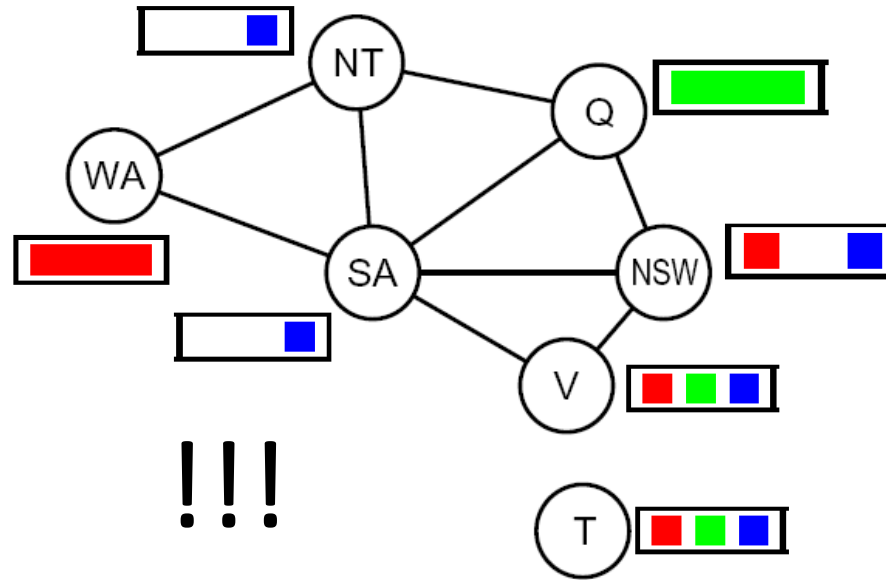
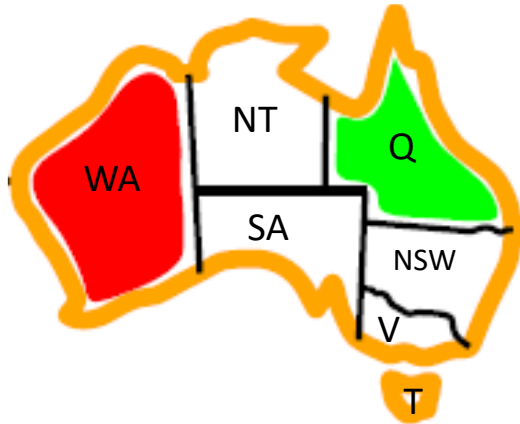
V->NSW

Q->NSW

SA->NSW

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

SA->NT

Q->NT

WA->SA

NT->SA

Q->SA

NSW->SA

V->SA

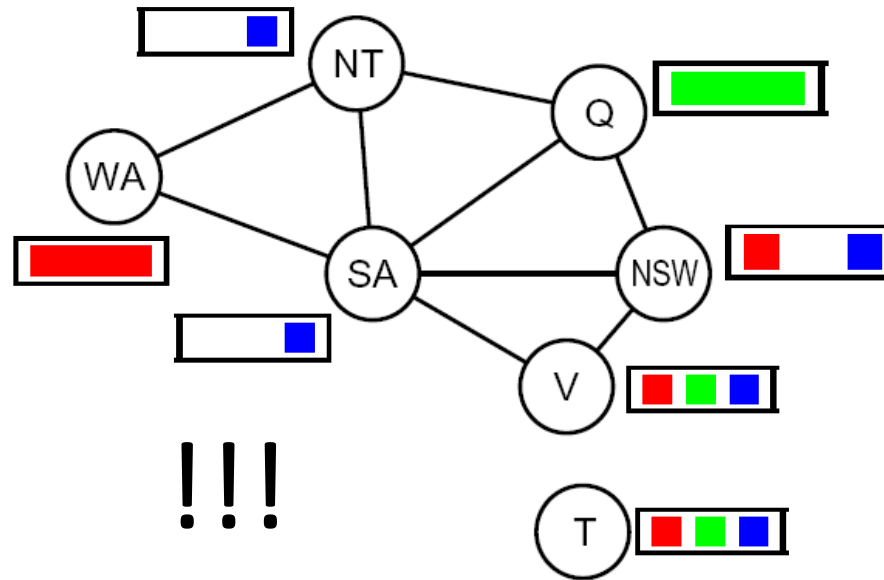
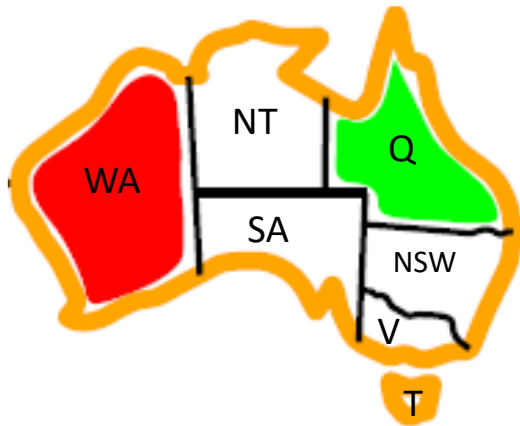
V->NSW

Q->NSW

SA->NSW

Remember: Delete from the tail!

AC-3: Enforce Arc Consistency of Entire CSP



Queue:

SA->NT

Q->NT

WA->SA

NT->SA

Q->SA

NSW->SA

V->SA

V->NSW

Q->NSW

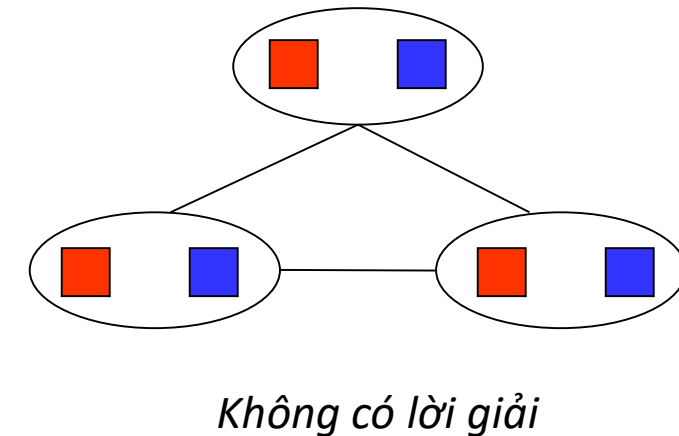
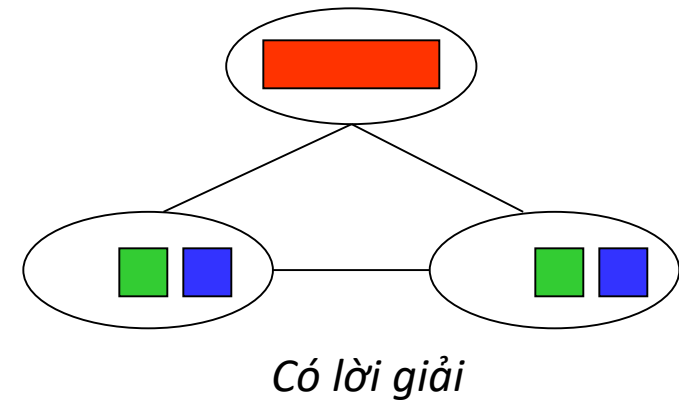
SA->NSW

- Quay lui lại bước gán Q
- “Arc consistency” phát hiện lỗi sớm hơn “forward checking”
- Có thể chạy độc lập hoặc sau mỗi bước gán

Remember: Delete from the tail!

Một số đặc điểm của AC-3

- Chỉ kiểm tra ràng buộc cục bộ:
 - Chỉ rút gọn miền giá trị, không chỉ ra lời giải
 - Có thể không còn lời giải nhưng không phát hiện được
- Có thể chạy bên trong vét cạn/quay lui
 - Thực hiện AC-3 sau mỗi bước gán giá trị
 - Cần lưu tập giá trị trước lúc để quay lui
- Có thể thực hiện độc lập như một bước tiền xử lý



Thuật toán Backtracking dùng AC-3

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

AC-3(*csp*)

- Thực hiện AC-3 ở bước nào?

Cải tiến: Thứ tự duyệt

- Chọn biến nào trước
- Chọn giá trị nào trước



Thứ tự: Biến có tập giá trị hợp lệ nhỏ nhất

- Thứ tự biến: Chọn biến có tập giá trị hợp lệ còn lại nhỏ nhất (Minimum remaining values - MRV)

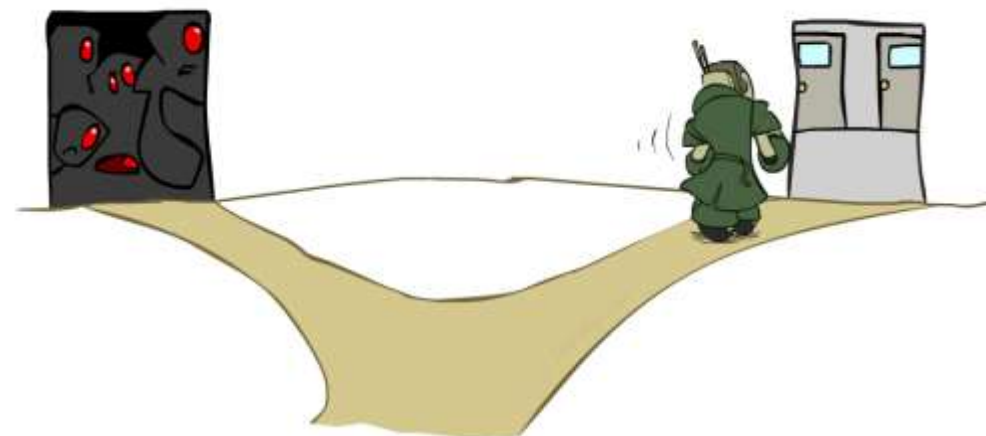
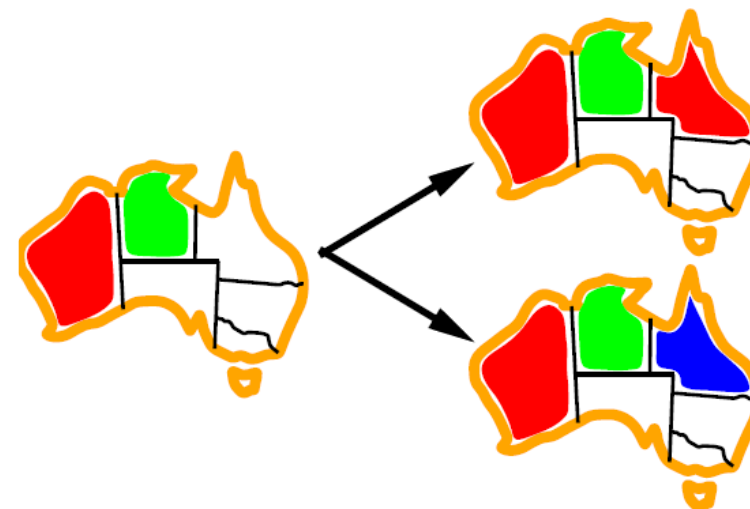


- Tại sao “min” tốt hơn “max”?
- Còn gọi là “biến bị ràng buộc nhiều nhất”



Thứ tự: Chọn giá trị ràng buộc ít nhất

- Thứ tự giá trị: Chọn giá trị ràng buộc ít nhất
 - Chọn giá trị mà sẽ loại trừ ít giá trị nhất của các biến còn lại
 - Có thể cần chạy “lọc” biến để xác định giá trị này
- Tại sao giá trị ít ràng buộc nhất lại tốt?

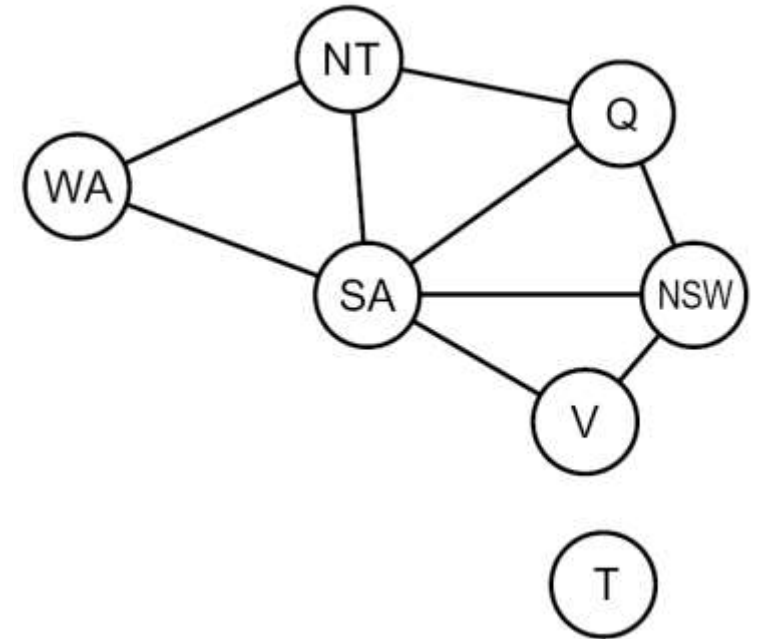


Cấu trúc bài toán

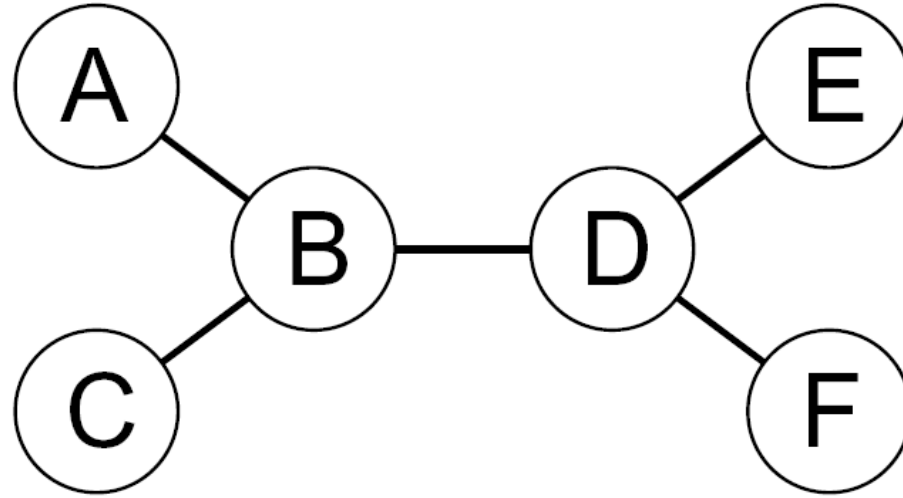


Cấu trúc bài toán

- Với CSP tổng quát, độ phức tạp trong trường hợp xấu nhất khi áp dụng backtracking là $O(d^n)$
 - n : số biến, d : số giá trị (trung bình)
- Nếu bài toán có cấu trúc đặc biệt, có thể giải hiệu quả hơn!
- Trường hợp phân rã được thành các bài toán con độc lập:
 - Tìm các miền liên thông
 - Giải với từng miền liên thông



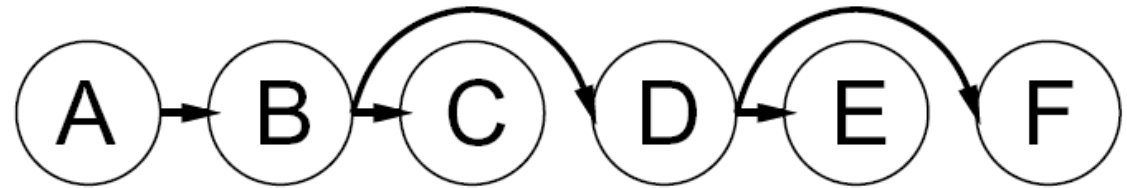
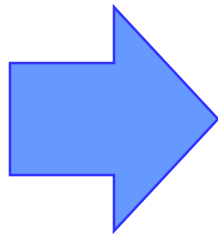
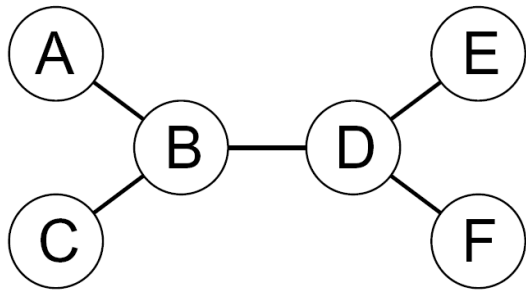
Trường hợp CSP có cấu trúc cây



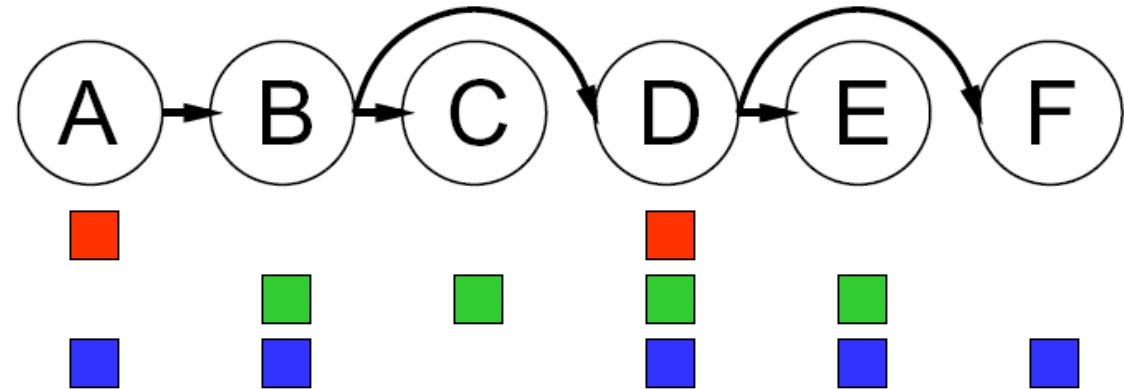
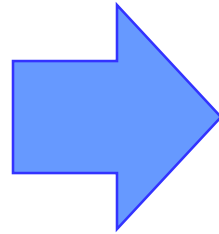
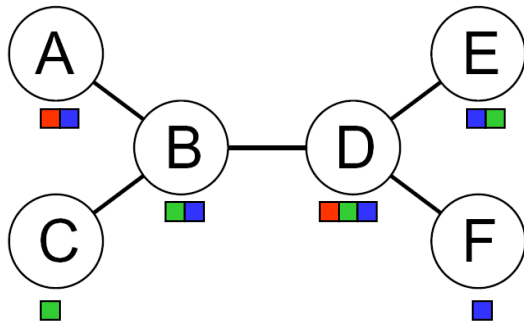
- Nếu đồ thị ràng buộc không có vòng lặp, độ phức tạp CSP là $O(nd^2)$
 - Nhỏ hơn CSP tổng quát với trường hợp xấu nhất là $O(d^n)$

CSP có cấu trúc cây

- Sắp xếp lại thứ tự thành cây: chọn nút gốc và đảm bảo thứ tự ràng buộc nút cha -> nút con

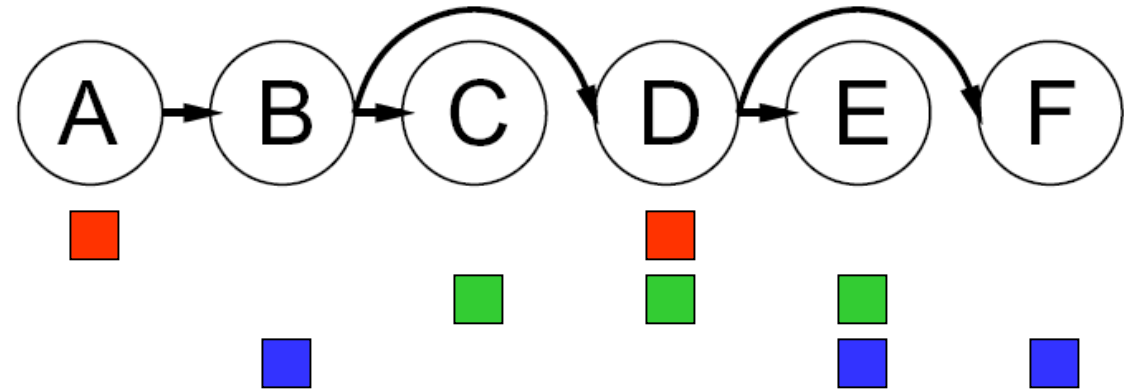
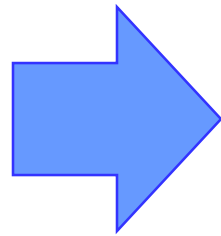
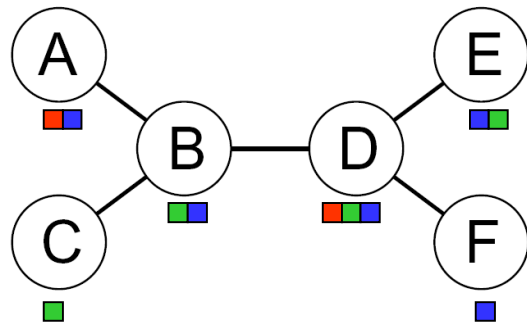


CSP có cấu trúc cây



- Pha (1) Xóa lùi: For $i = n : 2$, áp dụng $\text{RemoveInconsistent}(\text{Parent}(X_i), X_i)$

Tree-Structured CSPs



- Pha (2) Gán tiến: For $i = 1 : n$, gán X_i thỏa mãn $\text{Parent}(X_i)$
- Runtime: $O(nd^2)$
- Đầy đủ (tìm thấy lời giải nếu có)

Ràng buộc bậc cao

- Biến:

$F \ T \ U \ W \ R \ O \ X_1 \ X_2 \ X_3$

- Miền giá trị:

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- Ràng buộc:

$\text{alldiff}(F, T, U, W, R, O)$

$O + O = R + 10 \cdot X_1$

$\dots$

	T	W	O
+	T	W	O
F	O	U	R

Đồ thị ràng buộc

- Nút biến: thể hiện các biến
- Nút ràng buộc: nút hỗ trợ thể hiện các ràng buộc
- Cạnh: nối giữa nút ràng buộc và các nút biến tương ứng

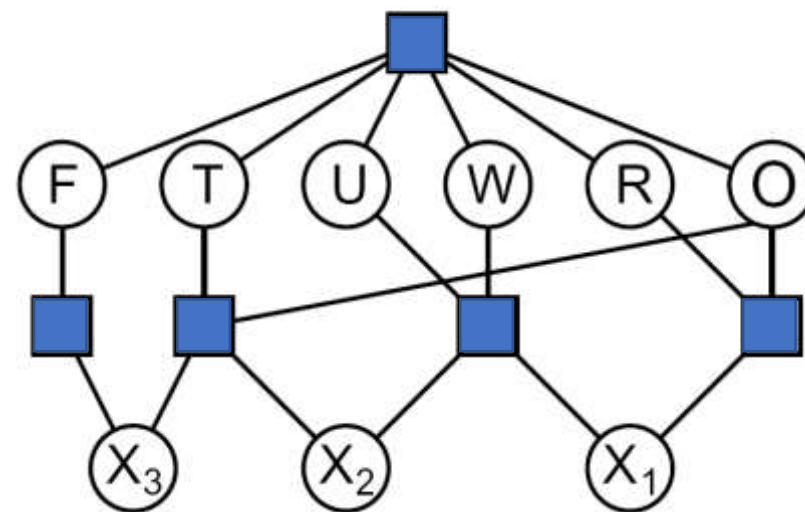
$$\begin{array}{r} \text{ T W O} \\ + \text{ T W O} \\ \hline \text{ F O U R} \end{array}$$

Ràng buộc:

$$\text{alldiff}(F, T, U, W, R, O)$$

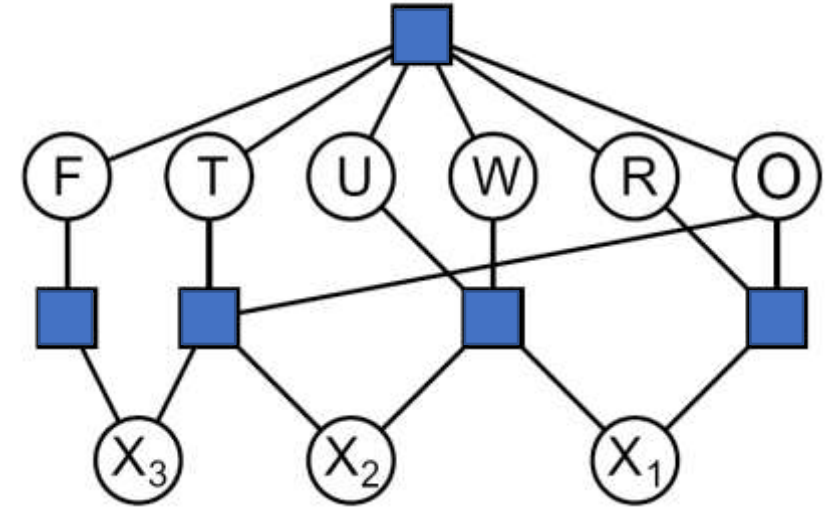
$$O + O = R + 10 \cdot X_1$$

...



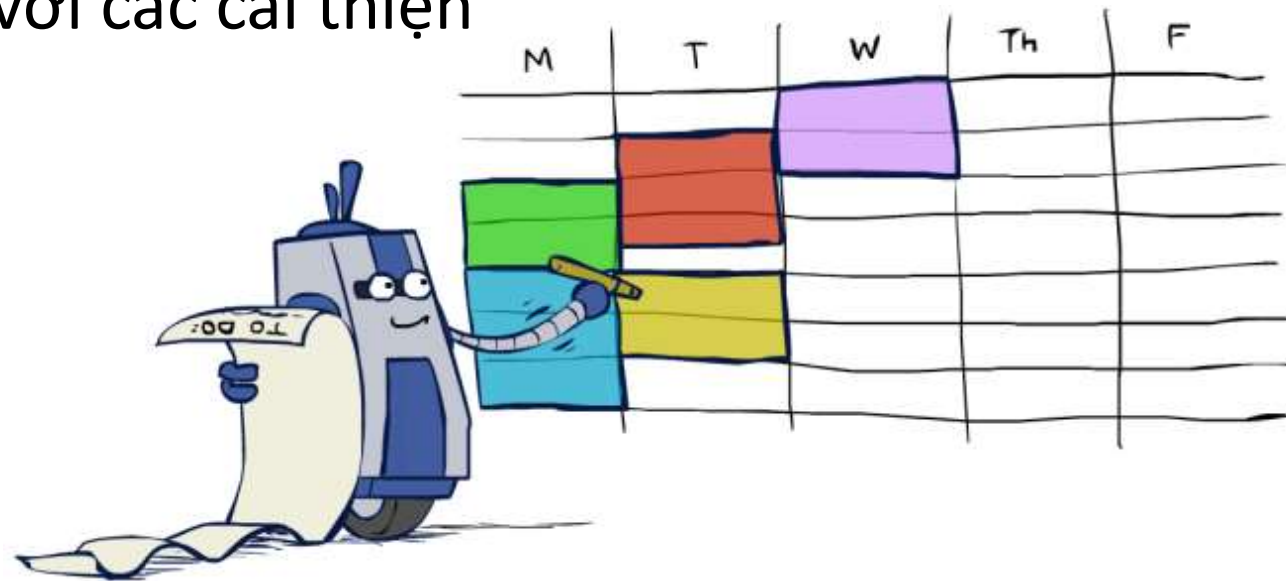
Giải bài toán ràng buộc bậc cao

- Có thể áp dụng vét cạn/quay lui, kiểm tra tiến, AC-3
- Cần cụ thể hóa từng loại ràng buộc với từng bài toán cụ thể



Tổng kết (I)

- CSP là một dạng bài toán tìm kiếm đặc thù
 - Mỗi trạng thái tương ứng một phép gán giá trị
 - Kiểm tra trạng thái đích = kiểm tra các ràng buộc của tập giá trị được gán
- Áp dụng tìm kiếm vét cạn/quay lui, với các cải thiện
 - Chọn thứ tự
 - Lọc giá trị (FC, AC-3)
 - Cấu trúc bài toán
- Các chiến lược khác
 - tìm kiếm/tối ưu cục bộ



Ví dụ: Crossword



LA Times, Sat, Sep 9, 2023 by David Karp / Ed. Patti Varol

Powered by [PuzzleMe™](#)

ACROSS

1 Luxuriates

6 Self starter?

9 Going concerns?

13 World record?

14 Express opp.

15 Hop on board?

DOWN

1 Baroque bigwig with a big wig

2 Friend of d'Artagnan

3 Put away noisily

4 Place with rounds and rounds of applause

5 Georgia, once: Abbr.

Ví dụ: Crossword



LA Times, Sat, Sep 9, 2023 by David Karp / Ed. Patti Varol

Powered by [PuzzleMe™](#)

ACROSS

36 Gearshift toppers

37 Zendaya's "Euphoria" role

38 Dorm style

39 "More soon"

42 Hebrides language

43 Super Bowl highlights?

44 Testicle

DOWN

30 Sanctioned

31 Vancouver-to-Yellowknife dir.

32 Nice resort area

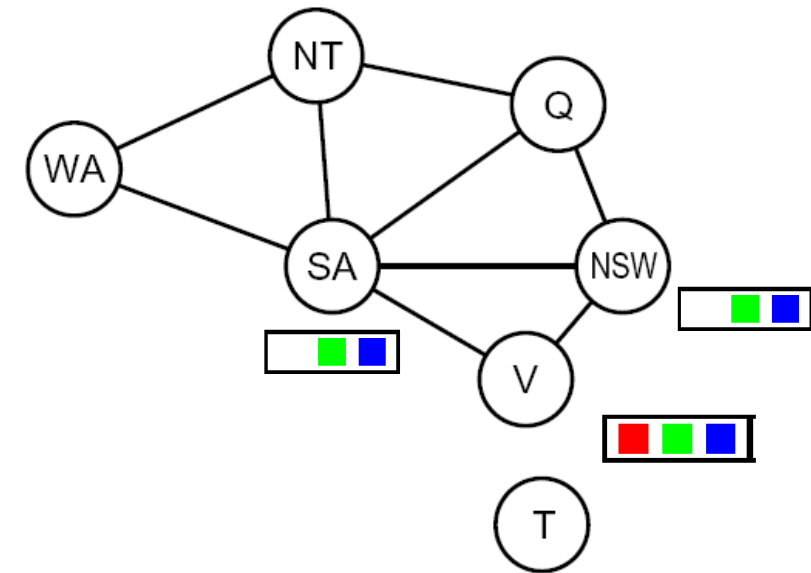
33 Pope of 903

34 Missouri River people

35 Fresh

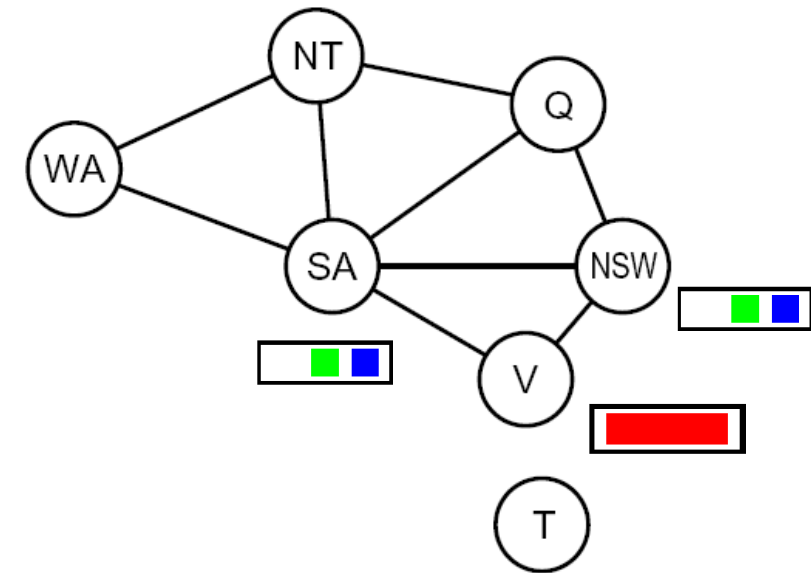
Tìm kiếm toàn cục đủ tốt chưa?

- Vét cạn/quay lui toàn cục với AC-3 vẫn có hạn chế là lặp lại các phép gán sẽ sinh lỗi (không trở thành lời giải)
 - AC-3 chỉ phát hiện mâu thuẫn trực tiếp (chỉ xét phù hợp 1 cạnh)
 - Cải tiến: phù hợp “chuỗi ràng buộc” (path consistency) !?
 - Kết quả các lần lọc trung gian không sử dụng lại được
- Với bài toán có độ phức tạp cao, thay vì tìm kiếm lời giải đầy đủ, có thể tạm thời tìm lời giải cục bộ (local search)
 - Giới hạn số ràng buộc, giới hạn độ sâu,...
 - Chia thành các bài toán con
- Áp dụng tư tưởng tìm kiếm cục bộ và “phù hợp chuỗi”
 - Lọc các giá trị chắc chắn không thành lời giải cục bộ của mọi nút
 - Tăng dần ràng buộc



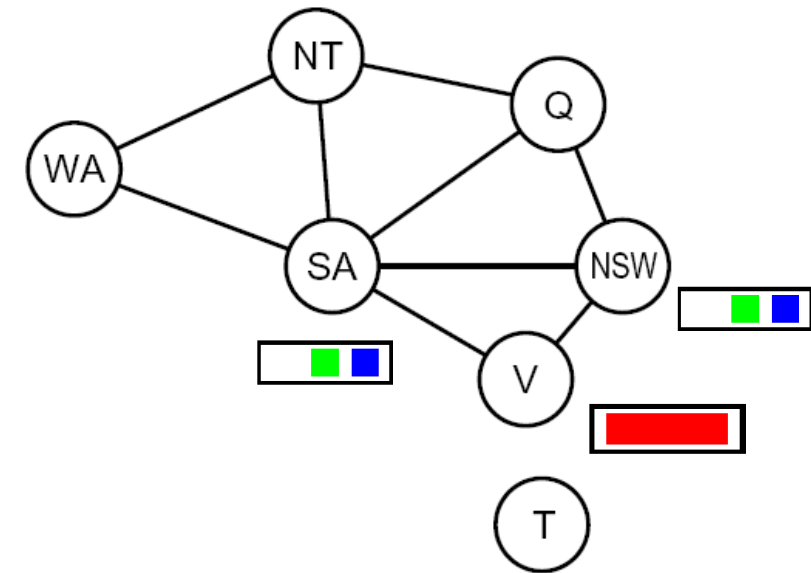
Phù hợp đỉnh

- Đỉnh X là phù hợp nếu mọi giá trị x_i của nó đều thuộc một lời giải
- Làm phù hợp đỉnh
 - Là phép lọc loại trừ các giá trị không thể gán cho lời giải nào
 - Bằng cách nào?
- Phù hợp đỉnh cục bộ
 - Đỉnh X phù hợp với độ sâu k khi với mọi tập k đỉnh liên thông chứa X thì mọi giá trị x_i của nó đều có thể thuộc một phép gán thỏa mãn ràng buộc với bộ k biến ứng với các đỉnh này



Lọc tăng dần (local search) cho CSP

1. c : tổng số cạnh, độ sâu $k = 2$
2. Hàng đợi Q chứa tất cả các đỉnh, lặp lại các bước sau khi Q chưa rỗng
 - i. Lấy một nút n từ Q
 - ii. Làm phù hợp đỉnh n với độ sâu k
 - iii. Nếu miền giá trị n được rút gọn thì thêm **các đỉnh liên quan đến n** vào Q
3. Tăng k đến $c-1$, lặp lại bước 2
4. Tìm lời giải đầy đủ bằng vét cạn/quay lui thông thường



Bài tập

- Viết chương trình giải đố Sudoku
- Viết chương trình giải Crossword với giả thiết biết tập từ ứng viên cho từng vị trí